FPT Education
FPT UNIVERSITY

MINISTRY OF EDUCATION AND TRAINING

FPT UNIVERSITY

Capstone Project Document

VISTA: A Vision-Language-Action System for Tabletop Manipulation with Robot Arms

<Group Name>	
Group Members	Nguyễn Gia Phát - 01 - SE190236 Nguyễn Minh Đức - 02 - SE190236 Nguyễn Lê Anh Quân - 03 - SE190236 Nguyễn Duy Sơn - 03 - SE192567
Supervisor	Mr. Lưu Giang Nam, Ph.D
Ext Supervisor	
Capstone Project code	FA2AI63

- Ho Chi Minh city, 12/2026 -

Table of Contents

Acknowledgement 4
Definition and Acronyms 4
List of tables 4
List of figures 5
I. Project Introduction 6
1. Overview 6
2. Project Background 7
3. Project Objective 9
4. Problem Statement 10
5. Significance of the Project 11
6. Project Scope & Limitations 12
II. Project Management Plan 14
1. Team Work 14
2. Project Management Approach 16
III. Existing Systems 29
1. Overview of the Field 29
2. Historical Context 31
3. Key Studies and Theories 31
4. Technological Advancements 34
5. Comparison of Existing Systems 35
6. Gaps in the Literature/Technology 39
7. Justification for the Project 42
IV. Methodology 45
IV. Methodology (sample 1) 45
1. Research Questions and Objectives 45
3. Data Collection and Preprocessing 61
4. Feature Selection and Engineering 64
5. Method 64
6. Model Training and Validation 64
7. Evaluation Metrics 65
8. Implementation Plan 65
9. Ethical and Safety Considerations 65
IV. Methodology (sample 2) 65

1. Project Objectives and RL Challenges....65
2. Environment Setup....65
3. Agent Design and States....66
4. Reward System....66
5. RL Algorithm....66
6. Training and Evaluation Strategy....66
7. Implementation Details....66
8. Ethical and Safety Considerations....66
V. System Design and Implementation....67
1. AI Model Integration....67
2. Data Flow and Processing....67
3. Deployment Strategy....67
4. Scalability and Maintenance....67
VI. Results and Discussion....68
1. Results and Analysis....68
2. Discussion....68
3. Recommendations....68
4. Personal Reflection (Optional)....68
VII. Conclusion....70
1. Summary of Findings....70
2. Contributions and Reflections on the Project....70
3. Limitations and Future Work....70
VIII. References....71
IX. Appendices....75

Acknowledgement

[Although not mandatory, a section thanking those who supported you during the project, including advisors, lecturers, friends, and family, can be included as an expression of gratitude.]

Definition and Acronyms

[A list of important terms and definitions used in the project. This ensures that all readers have a common understanding of the specialized terms used in the document.]

Acronym	Definition
AI	Artificial Intelligent
DL	Deep Learning
ML	Machine Learning
PM	Project Manager
VLA	Vision-language-action
VLM	Vision-language-model
ACT	Action-chunking-with-Transformers
RL	Reinforcement Learning
BC	Behaviour Cloning

List of tables

[A list of all the tables presented throughout the document.]

Table	Name	Context
1		

2		
3		

List of figures

[A list of all the figures presented throughout the document.]

Figure	Name	Context
1		
2		
3		

I. Project Introduction

[Provide the final project introduction information following the template as part II in the Report #1]

1. Overview

1.1 Project Information

[Provide the brief information of the project: project name, project code, group name, list out the details of the project team and supervisor (and stakeholders), etc.]

Project name : VISTA: A Vision-Language-Action System for Tabletop Manipulation with Robot Arms

Project code : FA2AI63

Group name: GFA26AI04

Project teams:

+ Nguyễn Gia Phát - SE190236
+ Nguyễn Minh Đức - SE
+ Nguyễn Duy Sơn - SE
+ Nguyễn Lê Anh Quân - SE

Supervisor: Mr.Giang-Nam Luu, Ph.D

1.2 Project Overview

[A brief overview of the project, including the main parts that will be discussed in the document, to give readers a general view of the report's structure and content.]

This project investigates *"Whether replacing the original vision-language model (**VLM**) backbone of **SmolVLA** (Small Vision-Language-Action) [26] with alternative lightweight VLMs can improve its performance in robotic manipulation while preserving its computational efficiency and accessibility?"*. The proposed approach focuses on evaluating alternative backbones with potentially stronger visual understanding capabilities and developing an efficient integration framework for incorporating them into the SmolVLA architecture. The central hypothesis is that an appropriately selected backbone can boost downstream manipulation performance without imposing disproportionate computational overhead.

To evaluate this hypothesis, experiments will be conducted using the low-cost SO-ARM101 robotic arm [14] and the LeRobot framework [6]. The modified SmolVLA variants (LFM2.5-VL-0.45B [18] and FastVLM-0.5B [29] as replacement backbones) will be compared against the original SmolVLA and Action Chunking with Transformers (ACT) [34] as a baseline on two tabletop manipulation tasks: Desk Stationery Decluttering (pilot task) and Halogen Lampshade Tray Packaging (secondary task). The evaluation will examine task performance and computational efficiency to determine whether backbone replacement provides measurable benefits under practical hardware constraints.

This report presents the project from initial analysis to experimental evaluation. It begins with project planning and management, followed by an analysis of existing systems and relevant technologies. The proposed methodology and system design are then described, including backbone selection, integration,

data collection, and model training. Subsequently, the implementation and experimental evaluation procedures are presented, followed by an analysis and discussion of the results. Finally, the report summarizes the main findings, acknowledges the project's limitations, and identifies directions for future work.

The final deliverables of this project will be :

1. **Capstone Project Report :** A complete written report covering the whole project: the problem statement, the proposed solution (the backbone-swapping framework and the selected VLM variants), training and evaluation details, analysis of results, discussion, and recommendations. All reported results will be reproducible from the documented setup, and every claim will be checked against the evaluation data or a cited source.
2. **Demonstration Videos :** To show that the selected policy works on real hardware, we will record two videos of the SO-ARM101 performing the 2 tasks
3. **Control and Monitoring App (prototype) :** A simple application that lets users: run a chosen policy on the SO-ARM101, switch between policies (the baseline and the variants), and watch the live camera stream together with a 3D visualization of the arm's current pose.

2. Project Background

[Introduce the context and setting of the project. This may include why the project was chosen, which problem the project aims to address, and the importance of solving this issue. This helps the reader understand the cause and purpose of the project.]

Before 2023, robotic manipulation research was largely confined to large, expensive hardware, creating a significant barrier to entry for the broader community. The introduction of the ALOHA system and the ACT (Action Chunking with Transformers) paper revolutionized the field by demonstrating that fine-grained manipulation tasks could be achieved with high success rates on low-cost, imprecise hardware. This breakthrough catalyzed a wave of community-driven development, leading to the proliferation of affordable teleoperation systems, large-scale open datasets, and scalable learning methods. Key outcomes of this movement include the LeRobot library, an open-source framework for the entire robot learning stack, and the SO-ARM101, an even more accessible robotic arm. As hardware becomes more affordable, policies like ACT are being superseded by newer, more lightweight models such as SmolVLA, which are designed to perform manipulation tasks more effectively [34], [6], [14].

Despite being a compact and efficient policy, SmolVLA suffers from significant limitations. The first primary disadvantage, explicitly noted in Section 5.1 of its own paper, is its **choice of VLM backbone**. The model relies on an off-the-shelf VLM, SmolVLM [19], which is pretrained mainly on document reading and OCR tasks. Consequently, it is hypothesized that SmolVLM is not proficient at localization, describing, or identifying objects within images—a critical capability for robotic manipulation. This limitation arose from a design trade-off: the authors of SmolVLA aimed to create a small, efficient, and community-driven VLA that drastically reduces training and inference costs. Unlike previous VLAs, which are typically massive (often with billions of parameters) and rely on academic or industrial datasets, SmolVLA's creators prioritized accessibility. As a result, their best and only option at the time was to use SmolVLM, the most compact and lightweight VLM available. These limitations are not necessarily inherent to

SmolVLA's overall architecture; rather, they motivate an investigation into whether alternative lightweight VLM backbones can improve its visual understanding and downstream manipulation performance[26].

This design choice reflects a broader **trade-off between computational efficiency and model capability**. SmolVLA was developed to make VLA-based robotic control more accessible by reducing training and inference costs while leveraging compact pretrained components and an open-source development ecosystem. However, selecting a compact backbone may also constrain the representational capacity available for visually demanding manipulation tasks. In its discussion of limitations and future directions, the SmolVLA paper highlights the importance of exploring how these architectures can be scaled further without sacrificing speed or accessibility. This observation motivates a central question in the present project: **Can a carefully selected alternative VLM backbone improve manipulation performance while preserving, or further improving, SmolVLA's computational efficiency?**[26]

As a result, this project focuses on two interconnected objectives: improving the visual-language capabilities of SmolVLA through backbone replacement and developing an efficient framework for integrating alternative VLMs into its architecture. Specifically, the project investigates whether replacing the original SmolVLM backbone with newer, lightweight VLMs can improve downstream robotic manipulation without undermining the efficiency and accessibility that characterize SmolVLA.

Since the release of the SmolVLA paper in June 2025, several new, reliable, and lightweight VLMs have been introduced that are better or even smaller than SmolVLM. These models are potential candidates for replacement. After careful consideration, we have selected LMF2.5-VL-0.45B and FastVLM-0.5B as the two suitable VLM models for testing and verifying the legitimacy of our solution, due to their lightweight characteristics and their inherent ability to process images effectively. By leveraging the low-cost SO-ARM101 hardware and the LeRobot library, we will evaluate our proposed solutions against the baselines (ACT and the original SmolVLA) on two tabletop manipulation tasks with different difficulty levels: Desk Stationery Decluttering and Halogen Lampshade Tray Packaging.

Importantly, this project is not limited to a straightforward plug-and-play replacement in which the original backbone is exchanged for another model without further adaptation. Instead, it aims to establish a systematic approach to backbone selection, integration, and evaluation, with the broader objective of improving the performance–efficiency trade-off of a lightweight VLA model. The investigation will examine the rationale for selecting the candidate backbones, compare their relevant architectural and computational characteristics, and develop an efficient integration framework for incorporating them into SmolVLA [18], [29].

The subsequent sections describe the proposed framework, data collection procedures, model training configurations, experimental design, and evaluation metrics. The experimental results will then be analyzed to determine whether backbone replacement produces meaningful improvements in manipulation performance and computational efficiency, and to identify the conditions under which these improvements can be achieved.

3. Project Objective

[The specific objectives of the project. This includes what the project hopes to achieve at the highest level. Objectives should be clear, measurable, and achievable. This guides the development of the project and the evaluation of its success.]

The primary objective of this project is to investigate whether replacing SmolVLA's original vision-language model (VLM) backbone with alternative lightweight VLMs can improve robotic manipulation performance while maintaining or enhancing computational efficiency. This investigation is motivated by two related challenges: the suitability of the original backbone's visual representations for manipulation tasks and the potential trade-off between model capability, computational cost, and scalability.

To address this, the project establishes the following specific objectives:

1. **Design and implement alternative backbone architectures.** Adapt and integrate LFM2.5-VL-0.45B and FastVLM-0.5B into the SmolVLA architecture as alternative VLM backbones. Develop the necessary integration and compatibility mechanisms, and produce *two trainable model variants* suitable for subsequent experimentation.
2. **Train the baseline and modified models.** Train and evaluate four model configurations: the original ACT baseline, the original SmolVLA baseline, SmolVLA with the LFM2.5-VL-0.45B backbone, and SmolVLA with the FastVLM-0.5B backbone. Record *training logs and relevant metrics*, including training loss, training duration, step execution time, and other available indicators of computational cost, to support reproducibility and comparative analysis.
3. **Evaluate manipulation performance and computational efficiency.** Conduct comparative experiments on the *Desk Stationery Decluttering* and Halogen Lampshade Tray Packaging. Measure task success rates and relevant efficiency metrics, including inference latency, memory consumption, and model parameter count where applicable. Analyze the results to determine whether backbone replacement provides measurable improvements over the original SmolVLA and *identify the best-performing configuration* under the defined evaluation criteria.
4. **Demonstrate real-world applicability.** Deploy the selected model on the SO-ARM101 robotic platform and investigate its suitability for a real-world halogen lampshade tray packaging task at a partner factory producing halogen light bulbs. Produce a video demonstration documenting the system's operation in the target environment and assess its practical limitations under real-world operating conditions.
5. **Develop a reusable backbone-integration strategy.** Document the architectural modifications, compatibility requirements, implementation procedures, and evaluation considerations involved in replacing SmolVLA's original VLM backbone. Based on the experimental findings, *formulate a systematic integration strategy* that can inform future investigations of alternative backbones and potentially support adaptation to other VLA architectures.
6. **Assess the potential for improving new lightweight VLA models (Highest-level).** Determine whether the proposed backbone modifications achieve a favorable balance between manipulation performance and computational efficiency. Compare the resulting models with the selected baselines and discuss the extent to which the *findings support further development of lightweight, accessible VLA systems.*

4. Problem Statement

[A specific statement of the problem that the project intends to solve. This section should detail the issue, including its causes, impacts, and specific challenges associated with solving the problem]

SmolVLA is a compact vision-language-action (VLA) policy of about 0.45B parameters. It is designed to be trained on a single GPU and deployed on consumer GPUs or even CPUs, while staying competitive with much larger VLAs on manipulation tasks. Despite this, the authors identify limitations that this project targets [26].

Limitation 1: an off-the-shelf backbone that is not suited to robotics. SmolVLA uses SmolVLM-2 as its backbone. SmolVLM-2 is a general-purpose VLM pretrained mainly on document understanding and OCR. Whether such a backbone is a good starting point for spatial relationships, affordances, and physical interaction is an open question the authors raise themselves. SmolVLA also keeps its visual input minimal: it disables image tiling and uses only 64 visual tokens per frame, which may sacrifice fine detail on small objects. Together, these choices make perception and object localization in cluttered scenes a plausible weak point [26].

Limitation 2: little room to improve without adding weight. The authors list further model-size scaling as future work. SmolVLA already uses a very small VLM and only a subset of its early layers. Gains from the backbone therefore cannot come from simply making the model larger without giving up the efficiency that defines it [26].

Why do these limitations exist? When SmolVLA was released (June 2025), SmolVLM was the best accuracy-per-memory option among small VLMs. Its smaller variants need roughly 0.8–1.2 GB of RAM at batch size 1, compared with about 27.7 GB for *MolmoE-A1B-7B*, and it matched or beat comparable models such as *InternVL2-2B* [21] on several benchmarks. Under a tight efficiency budget, there was little reason to choose another backbone. Because the architecture was then tuned around this backbone, it left little room for change [19].

What has changed. Newer compact VLMs now offer a possible alternative: FastVLM (CVPR 2025, 0.5B variant), LFM2-VL (450M and 1.6B), and Qwen3.5-0.8B [23] (released in early 2026). Their training data and architectures differ from SmolVLM's, so they may provide better visual grounding at a similar scale [29], [2].

Problem statement. *Can carefully selected alternative VLM backbones improve the manipulation performance of SmolVLA while preserving its computational efficiency and practical deployability?*

Challenges.

1. *Backbone selection.* Candidates must stay lightweight, and their pretraining should emphasize natural-image understanding and grounding rather than documents.
2. *Layer selection.* SmolVLA uses only part of its VLM's layers, so the best layers to keep must be re-determined for each new backbone. Some candidates use non-standard layer types (for example, LFM2-VL's hybrid convolution-attention backbone), which complicates this.

3. *Integration.* There is no ready-made, tested way to swap the backbone, so we must build the adapter and training pipeline ourselves, covering token and feature dimensions, the action-expert connection, and the training setup.
4. *Uncertain outcome.* A new backbone may not remove either limitation. Conclusions will rest on training and evaluating each variant against the original SmolVLA, in simulation and on real tasks, measuring both success rate and efficiency (parameters, memory, latency).

5. Significance of the Project

[The importance of the project. This is where you describe the benefits that the project brings, both theoretically and practically, and its impact on the scientific community, industry, or society at large.]

SmolVLA was created to make vision-language-action (VLA) research more accessible. Most VLAs have billions of parameters and are trained on academic or industrial datasets. SmolVLA is a compact policy trained on community-collected data. It can be trained on a single GPU and deployed on consumer hardware, while remaining competitive with much larger models on manipulation tasks. The limitations described in Section 4 (a backbone pretrained mainly on documents and OCR, and little room to improve without adding weight) restrict how far this approach can go. This project addresses those limitations, and its significance lies in three areas [26].

Scientific significance. The project provides an empirical answer to an open question raised by SmolVLA's authors: whether a VLM backbone better suited to visual grounding than the original one improves robotic manipulation at a comparable scale. Whatever the outcome, the results are informative. Gains would show that backbone choice matters in lightweight VLAs.

Practical and industrial significance. The project is implemented and evaluated on the low-cost SO-ARM101 robot arm through a Halogen Lampshade Tray Packaging task. If the improved variants perform reliably, the results will suggest that affordable hardware paired with a compact policy can handle simple, repetitive sorting workflows. That would give small and medium-sized factories a cheaper option than expensive industrial robots.

Community and methodological significance. The most reusable result is the framework itself: a documented strategy for choosing a replacement backbone, selecting which of its layers to use, and adapting it into SmolVLA for training and evaluation. Today, swapping a backbone in a VLA requires custom engineering each time. A tested procedure lowers that barrier. Researchers with stronger hardware can apply the same process to larger VLM backbones, and those with limited resources can test small ones. It also helps keep progress on lightweight VLAs open to people without large GPU clusters.

6. Project Scope & Limitations

[The scope of the project as well as its limitations. This includes what the project will and will not cover. This section helps set expectations and prevent misunderstandings.]

What the project will cover : (1) Backbone replacement. We will replace SmolVLA's SmolVLM-2 backbone with a small number of lightweight VLMs. The candidates are drawn from the models in Section 4 (Qwen3.5-0.8B, FastVLM, and LFM2-VL or its newer LFM2.5-VL version). The final list will depend on how well each can be integrated and how much compute is available. **(2) Swapping framework.** We will build and document a procedure for swapping the backbone. It covers choosing the layers to keep, matching feature dimensions to the action expert, and setting up training so that each variant can be trained and tested. **(3) Data collecting.** As for the calibration on each hard-ware is different due to human stochastic in action and we do not use any stimulations, we will follow the design in experiment protocol of SmolVLA paper to collect 300 demonstrations for each task "*Desk Stationery Decluttering"* (pilot task) and "Halogen Lampshade Tray Packaging*"* (secondary task)*.* **(4) Training.** Each variant and the original SmolVLA baseline will be trained under the same data and settings, so the comparison is fair. **(5) Evaluation.** The variants and baseline will be compared on the desk decluttering and Halogen Lampshade Tray Packaging task using the SO-ARM101 arm. We will measure task success rate. We will also measure efficiency: parameter count, memory use, and inference latency. **(6) Analysis.** We will analyze which backbone properties (pretraining data, vision encoder, layer choice) appear to relate to performance differences, and report limitations of the framework.

What the project will not cover : (1) Designing a new VLA architecture or modifying the action expert, other than the changes needed to connect a new backbone. **(2)** Large-scale pretraining on community datasets, or training backbones from scratch. **(3)** Cross-embodiment evaluation, long-horizon tasks, or tasks other than Halogen Lampshade Tray Packaging. **(4)** Larger VLM backbones (>1B parameter), or any model where efficiency requirements no longer apply. **(5)** Deployment engineering for production use, such as safety certification, long-term reliability testing, or integration with factory systems. **(6)** Fully eliminating the two limitations in Section 4. We aim to measure how much backbone replacement helps, not to prove it solves them.

Limitations of this project :

- **Small candidate set.** Only a few backbones can be tested, so a better-suited model may exist that we do not try.
- **Limited data and trials.** Training and evaluation data will be small compared with the original SmolVLA pretraining data, and real-world trial counts will be limited by time.
- **Single task and robot.** Results come from one task on the low-cost SO-ARM101 arm, whose payload and precision are limited. They indicate feasibility for light sorting tasks only.
- **Compute limits.** Hyperparameter tuning and training length are restricted by available GPU resources, so reported performance may not reflect each backbone's best possible result.
- **Fast-changing field.** Newer lightweight VLMs may appear during the project, and the candidate list will be fixed at a defined point.

II. Project Management Plan

[Provide the final project plan information following the template as part II in the Report #2]

1. Team Work

1.1 Team Structure and Roles

[Describe the structure of the team and the roles of each member. This should include information about specific tasks, responsibilities, and communication methods between members.]

Member	Role	Responsibilities
Nguyễn Gia Phát	Project Manager	Divides tasks, builds and updates the plan, tracks milestones, and leads research on the proposed method. Main contact for the supervisor and the factory partner. Compiles the final report.
Nguyễn Minh Đức	AI Engineer	Trains the policies, implements the backbone-swapping method in code, and develops the control and monitoring app.
Nguyễn Lê Anh Quân	AI Engineer	Reviews and tests the code, tunes policy hyperparameters, maintains the result logs, analyses metrics for insights, and supports app testing.
Nguyễn Duy Sơn	Data Engineer	Collects, cleans, and annotates data for both tasks. Prepares the test setup (robot, camera, task layout), runs the real-robot trials with Quân, records the demonstration videos, and supports research and verification of proposals.

Table 1.

Every member writes the report sections that cover their own work, and Phát compiles and checks the final document. The team uses Zalo, Gmail, and Discord. In-person meetings are held twice a week, on Wednesday and Sunday. Each member reports completed work within the same day so progress can be checked daily

1.2 Communication Plan

[Describe how and how often project information will be communicated among team members and advisor (and other stakeholders).]

Who	How often	Purpose
Project Manager and team	Daily (end of day)	Check that each member's tasks are done and spot problems early.
Whole team	Twice a week (Wednesday, Sunday)	Held direct, in-person meeting. Review deliverables and milestones, and adjust the plan in time.
AI Engineers	Daily	Record work in the worklog. Report to the Project Manager when a training run finishes, or immediately when a problem requires a change to the plan.
Data Engineer	Daily	Report collection and cleaning progress in the daily report. Notify the AI Engineers when a dataset version is ready, and the Project Manager if collection is blocked or data quality affects the schedule.
Factory partner	Once a week	The Project Manager checks for changes or requests and confirms the testing schedule. Before on-site testing, the team confirms site access, safety rules, and permission to record video. New requests follow the change-control rule in Section 6.
Supervisor	Once a week	Ask for advice on ideas, plans, and methods, report progress, and take feedback. The supervisor also decides, together with the team, whether a new factory request fits the scope.

Table 2. …

2. Project Management Approach

2.1 WBS

[Provide detailed descriptions for each task or duty within the WBS, including objectives, required resources, and assigned responsibilities.]

Table 3. …

WBS ID	Work package / task	Objective & description	Deliverable / evidence to attach	Required resources	Owner	Support	Start	Days	End	Pred.	Report Obj.
1	**Project management & governance**	**Plan, track and control the project; keep evidence for every task.**					**15-Sep**	**77**	**30-Nov**		
1.1	Kickoff, roles, tools & repository setup	Agree roles and communication rules; create the Git repo, shared drive, worklog and meeting-minute templates.	Role table; repo and worklog templates	GitHub, Zalo, Gmail, Discord	Phát	All	15-Sep	4	18-Sep	-	PM
1.2	Project plan: WBS, schedule & risk register	Break the work into work packages with owners, dates and deliverables; identify the top risks.	This WBS; risk register; Report Section II	Excel, report template	Phát	All	15-Sep	8	22-Sep	1.1	PM
1.3	Progress tracking & meetings	Daily worklog; team meetings (Wed, Sun); weekly supervisor and factory updates. Attach evidence (logs, photos, run IDs) to every finished task.	Worklog; meeting minutes; evidence log	Worklog, Discord	Phát	All	15-Sep	77	30-Nov	1.1	PM
1.4	Risk & change-control log	Update the risk register weekly; log every scope or schedule change with its impact and the supervisor's decision.	Risk register; change log	Excel	Phát	Quân	22-Sep	70	30-Nov	1.2	PM

1.5	Project closure & lessons learned	Compare plan vs actual, record lessons learned, archive code, data and results.	Closure summary; archived repo and data	Git, shared drive	Phát	All	24-Nov	7	30-Nov	10.12	PM
2	**Research & requirements**	**Define the task and build the literature base for the study.**					**15-Sep**	**24**	**08-Oct**		
2.1	Task definition & factory requirements	Specify the sorting task: which objects, how many, layout, belt speed, lighting, success criteria. Confirm with the partner factory.	Task specification sheet; partner confirmation	Factory contact, SO-ARM101 reach data	Phát	Sơn	15-Sep	10	24-Sep	-	O4
2.2	Literature search & screening	Run a documented search (sources, queries, inclusion/exclusion) and build the reference library.	Search log; reference library; screened paper list	arXiv, Google Scholar, Zotero	Phát	Sơn	15-Sep	12	26-Sep	-	O6
2.3	Architecture study: ACT, SmolVLA, LeRobot, SO-ARM101	Analyse prior architectures: inputs and outputs, data flow, method, reported results, strengths and limits.	Architecture notes with diagrams, formulas, sources	Papers, LeRobot code	Đức	Quân	17-Sep	10	26-Sep	-	O1
2.4	Candidate backbone survey & selection	Compare lightweight VLMs on size, pretraining data, layer structure, licence and speed. Shortlist LFM2.5-VL-0.45B and FastVLM-0.5B.	Selection table with criteria and decision	Hugging Face model cards, papers	Đức	Quân	24-Sep	10	03-Oct	2.3	O1
2.5	Concept matrix & gap analysis (Section III draft)	Compare existing systems on common criteria and derive the research gap from the matrix.	Concept matrix; Section III draft	Reference library	Phát	Đức	29-Sep	10	08-Oct	2.2, 2.4	O6
3	**Methodology & experimental design**	**Fix research questions, metrics, baselines and the evaluation protocol before the final runs.**					**24-Sep**	**19**	**12-Oct**		

3.1	Research questions, hypotheses & success criteria	State the research question, hypothesis and numeric pass criteria for each objective.	RQ and hypothesis list; success-criteria table	-	Phát	Quân	24-Sep	7	30-Sep	2.1	O6
3.2	Metrics selection & justification	Choose success rate, progress score, latency, memory and parameter count. Give formulas and explain why other metrics are unsuitable.	Metrics specification with formulas	Papers	Quân	Phát	29-Sep	7	05-Oct	3.1	O3
3.3	Baseline selection & reproduction	Set up ACT and original SmolVLA in LeRobot (at least 2 baselines) and compare with numbers published in other studies.	Baseline configs; comparison table vs published results	LeRobot, Kaggle GPU	Đức	Quân	29-Sep	12	10-Oct	2.3	O2
3.4	Method rationale: formulas, citations, noise space	Document the formulas of the chosen method, including the flow-matching noise space: why it is used and how it applies in the real setting.	Method rationale note with formulas and sources	SmolVLA and flow-matching papers	Đức	Phát	01-Oct	8	08-Oct	2.3	O1
3.5	Evaluation protocol & statistics plan	Define trial counts (with sample-size reasoning), randomized start positions, success definitions, confidence intervals and tests.	Evaluation protocol document	Python (SciPy, statsmodels)	Quân	Phát	03-Oct	9	11-Oct	3.1, 3.2	O3
3.6	◆ Protocol freeze	Protocol and metrics frozen before the final trials.	Dated, versioned protocol	-	Phát	All	12-Oct	1	12-Oct	3.5	O3
4	**Data collection & preprocessing**	**Build a documented, diverse dataset for both tasks.**					**22-Sep**	**39**	**30-Oct**		

4.1	Hardware setup & calibration	Set up leader/follower SO-ARM101, cameras and task layouts; calibrate and record the setup.	Calibration files; setup photos; layout diagram	SO-ARM101, cameras, mock-up	Sơn	Quân	22-Sep	10	01-Oct	2.1	O2
4.2	Dataset specification & collection protocol	Define data source (self-collected vs public), number of samples, features, variables X/Y/Z, storage format and diversity plan.	Dataset datasheet; collection protocol	LeRobot dataset format	Sơn	Quân	29-Sep	7	05-Oct	4.1	O2
4.3	Pilot collection & quality check	Collect pilot episodes; check synchronization, image quality and episode lengths; fix the protocol.	Pilot report with sample videos	SO-ARM101, cameras	Sơn	Đức	05-Oct	5	09-Oct	4.2	O2
4.4	Collect demonstrations – Task A (Desk Stationery Decluttering)	Teleoperate and record the target of 300 demonstrations with varied object positions.	Task A dataset; collection log	SO-ARM101, cameras	Sơn	Quân	10-Oct	8	17-Oct	4.3	O2
4.5	Collect demonstrations – Task B (Halogen Lampshade Tray Packaging)	Teleoperate and record the target of 300 demonstrations with varied object positions and belt conditions.	Task B dataset; collection log	SO-ARM101, cameras, mock-up	Sơn	Quân	18-Oct	8	25-Oct	4.4	O2
4.6	Data cleaning, QC, episode-level split & statistics	Remove failed or corrupt episodes, split by episode, compute dataset statistics (size, depth, breadth, diversity).	Cleaning report; split files; statistics tables	Python	Sơn	Quân	14-Oct	16	29-Oct	4.4	O2
4.7	◆ Dataset v1.0 frozen	Dataset version locked before any final training run.	Versioned dataset with hash	-	Sơn	Quân	30-Oct	1	30-Oct	4.6	O2
5	**Model integration (backbone replacement)**	**Integrate the alternative VLM backbones into SmolVLA.**					**01-Oct**	**50**	**19-Nov**		

5.1	Integration design	Design the adapter: token and feature dimensions, which layers to keep for each backbone, connection to the action expert.	Integration design doc with diagram	LeRobot code	Đức	Quân	01-Oct	10	10-Oct	2.4, 3.4	O1
5.2	Implement LFM2.5-VL-0.45B variant	Implement the backbone and adapter in LeRobot; handle its hybrid convolution-attention layers.	Working code branch; config file	LeRobot, Hugging Face	Đức	Quân	08-Oct	12	19-Oct	5.1	O1
5.3	Implement FastVLM-0.5B variant	Implement the backbone and adapter in LeRobot.	Working code branch; config file	LeRobot, Hugging Face	Đức	Quân	11-Oct	12	22-Oct	5.1	O1
5.4	Sanity tests, parameter count & complexity	Forward-pass and small-subset overfit tests; count parameters (millions), estimate complexity, assess ways to reduce parameters.	Test log; parameter and complexity table	PyTorch profiler	Quân	Đức	20-Oct	8	27-Oct	5.2, 5.3	O1
5.5	Reusable integration strategy documentation	Write the swap procedure, compatibility checklist and lessons for future backbones.	Integration guide; material for Sections V and VII	-	Đức	Phát	10-Nov	10	19-Nov	5.4	O5
6	**Training**	**Train baselines and variants under identical data and settings.**					**20-Oct**	**32**	**20-Nov**		
6.1	Training configuration & hyperparameter justification	Set steps, batch size, learning rate and seeds; justify each choice from papers or calculations; plan Kaggle GPU sessions.	Config table with sources; Kaggle session plan	Kaggle GPU	Quân	Đức	20-Oct	8	27-Oct	3.3, 5.1	O2
6.2	Train baselines: ACT & original SmolVLA	Train on the frozen dataset with the common settings; log steps, runtime and GPU.	Checkpoints; training logs	Kaggle GPU	Quân	Đức	31-Oct	9	08-Nov	6.1, 4.7	O2

6.3	Train SmolVLA + LFM2.5-VL-0.45B	Train with the same data and settings as the baselines.	Checkpoints; training logs	Kaggle GPU	Đức	Quân	01-Nov	12	12-Nov	6.1, 4.7, 5.4	O2
6.4	Train SmolVLA + FastVLM-0.5B	Train with the same data and settings as the baselines.	Checkpoints; training logs	Kaggle GPU	Đức	Quân	03-Nov	12	14-Nov	6.1, 4.7, 5.4	O2
6.5	Fine-tuning improvement round	Try tuning directions on the best variant (for example layers kept, learning-rate schedule, image tokens); record before/after metrics for each direction.	Before/after comparison table	Kaggle GPU	Quân	Đức	13-Nov	8	20-Nov	6.3, 6.4	O2
6.6	Training log & reproducibility pack	Collect loss curves, steps, runtime, GPU configuration, seeds and commit hashes for every run.	Reproducibility pack	local logs, Git	Quân	Đức	05-Nov	16	20-Nov	6.2	O2
6.7	◆ Training complete	All final checkpoints available.	Checkpoint list	-	Quân	Đức	20-Nov	1	20-Nov	6.5	O2
7	**Evaluation**	**Measure manipulation performance and efficiency with the frozen protocol.**					**09-Nov**	**19**	**27-Nov**		
7.1	Offline validation on held-out episodes	Compute action-prediction error on held-out episodes for all four models as an early quantitative signal.	Offline results table	Python	Quân	Đức	09-Nov	7	15-Nov	6.2	O3
7.2	Efficiency benchmarking	Measure latency (median and p95, warm-up excluded), memory use and parameter count on the target hardware.	Efficiency table	Target PC/GPU, profiler	Quân	Đức	13-Nov	5	17-Nov	6.3, 6.4	O3

7.3	Real-robot trials: baselines	Run the planned trials for ACT and original SmolVLA; log every trial and record video.	Trial log; videos	SO-ARM101, cameras	Sơn	Quân	09-Nov	7	15-Nov	6.2, 3.6	O3
7.4	Real-robot trials: backbone variants	Run the planned trials for both variants under the same protocol; log every trial and record video.	Trial log; videos	SO-ARM101, cameras	Sơn	Quân	14-Nov	8	21-Nov	6.3, 6.4, 3.6	O3
7.5	Re-evaluate the tuned best model	Repeat the trials for the model from the fine-tuning round.	Trial log; videos; before/after results	SO-ARM101, cameras	Sơn	Quân	21-Nov	5	25-Nov	6.5	O3
7.6	Statistical & failure-case analysis	Compute confidence intervals and tests; classify failures (perception, grasp, timing, hardware); select qualitative cases.	Statistics tables; failure taxonomy; qualitative examples	Python	Quân	Phát	22-Nov	6	27-Nov	7.3, 7.4	O3, O6
8	**Control & monitoring app (prototype)**	**Let users run a policy, switch between policies and watch the camera plus 3D arm pose.**					**12-Oct**	**32**	**12-Nov**		
8.1	App requirements & architecture	List functions, interfaces and the data flow between app, policy and robot.	Requirements list; architecture diagram	-	Đức	Phát	12-Oct	6	17-Oct	2.3	O4
8.2	Policy runner & policy switching	Run a chosen policy on the SO-ARM101 and switch between baseline and variants.	Working module	Python, LeRobot	Đức	Quân	23-Oct	10	01-Nov	8.1, 5.4	O4
8.3	Live camera stream & 3D arm-pose visualization	Show the live camera feed together with the arm's current pose in 3D.	Working module	Python, 3D viewer library	Quân	Đức	28-Oct	10	06-Nov	8.1	O4

8.4	App integration test on SO-ARM101	Test the whole app on the real robot; fix defects.	Test report; app prototype	SO-ARM101, cameras	Quân	Đức	07-Nov	6	12-Nov	8.2, 8.3	O4
9	**Demo & real-world deployment**	**Show the selected policy on real hardware, including failure cases.**					**12-Nov**	**17**	**28-Nov**		
9.1	Demo setup & safety check	Check emergency stop, joint limits and workspace; confirm site access and recording permission.	Safety checklist; permission confirmation	SO-ARM101, factory contact	Sơn	Phát	12-Nov	4	15-Nov	2.1	O4
9.2	Demo dry-run for Review 3 & fixes	Run the full demo with the best model so far; fix the problems found.	Dry-run video; defect list	SO-ARM101, app	Quân	Sơn	16-Nov	4	19-Nov	8.4, 9.1	O4
9.3	On-site factory trial & demo videos	Record the best variant and the original SmolVLA on the Halogen Lampshade Tray Packaging task, including failure cases.	Two demo videos; on-site test notes	SO-ARM101, factory, camera	Sơn	Phát, Quân	25-Nov	4	28-Nov	7.5, 9.1	O4
10	**Reporting, reviews & submission**	**Write and verify the report; prepare the Review 2 and Review 3 evidence.**					**06-Oct**	**56**	**30-Nov**		
10.1	Report: Sections I–II (final)	Final introduction and project management plan; remove inconsistencies in scope, tasks and models.	Sections I–II	Report template	Phát	All	06-Oct	14	19-Oct	1.2, 3.1	PM
10.2	Report: Section III – existing systems	Write literature review from the concept matrix and search log.	Section III	Reference library	Phát	Đức	08-Oct	10	17-Oct	2.5	O6

10.3	Report: Section IV – methodology	Write research design, data, training setup, protocol, statistics and threats to validity.	Section IV	Report template	Quân	Phát	12-Oct	14	25-Oct	3.6, 4.2	O2, O3
10.4	Review 2 evidence pack & slides	Collect roadmap, dataset specification, architecture analysis, baselines, metrics and current numbers.	Review 2 slides; evidence folder	Slides, evidence log	Phát	All	20-Oct	7	26-Oct	1.3	PM
10.5	◆ Review 2 (27/10/2026)	Review meeting with the reviewer.	Review 2 form with comments	-	Phát	All	27-Oct	1	27-Oct	10.4	PM
10.6	Review 2 feedback → action plan & WBS update	Turn each reviewer comment into tasks, owners and dates; update this WBS and the risk register.	Action list; updated WBS	Excel	Phát	All	28-Oct	3	30-Oct	10.5	PM
10.7	Report: Section V – system design	Write architecture, data flow, interfaces, deployment and latency budget.	Section V	Report template	Đức	Quân	03-Nov	10	12-Nov	5.4, 8.1	O1, O4
10.8	Review 3 evidence pack & slides	Collect results tables, failure cases, tuning before/after and demo video.	Review 3 slides; evidence folder	Slides, evidence log	Phát	All	14-Nov	5	18-Nov	7.4	PM
10.9	◆ Review 3 (date to be confirmed)	Review meeting with the reviewer. Date is a placeholder until confirmed.	Review 3 form with comments	-	Phát	All	20-Nov	1	20-Nov	10.8	PM
10.1	Report: Section VI – results & discussion	Present results with n and confidence intervals; give each objective a verdict; discuss limits and threats.	Section VI	Report template	Phát	Quân	19-Nov	9	27-Nov	7.4, 7.6	O3, O6

10.11	Report: Conclusion, references, appendices, front matter	Write Section VII, finalize references, appendices, abstract, acronym list, lists of tables and figures.	Sections VII–IX; front matter	Zotero, report template	Phát	Sơn	23-Nov	6	28-Nov	10.10	PM
10.12	Claim-ledger verification & quality gates (G1–G3)	Check every claim against its source or run log; a different person verifies than the one who wrote it.	Verified claim ledger; gate checklist	Spreadsheet ledger	All	Phát	22-Nov	7	28-Nov	10.7, 10.10	PM
10.13	Final proofreading & report freeze	Language, formatting and number-consistency check.	Final report PDF/DOCX	Report template	Phát	All	28-Nov	2	29-Nov	10.11, 10.12	PM
10.14	*Final submission*	Submit the report, videos and app.	Submitted package	-	Phát	All	30-Nov	1	30-Nov	10.13	PM

2.2 Risk Management

[Describe potential risks during the project execution and preventive measures or response plans for each risk.]

Risk management is conducted through the identification, assessment, mitigation, and monitoring of potential events that could affect the project's schedule, resources, or deliverables. The risks are evaluated according to their likelihood of occurrence and potential impact on project completion. Preventive measures are established to reduce the probability of occurrence, while contingency plans are prepared to minimize disruption if a risk materializes. Three major risks have been identified for this project:

Risk 1: Training delays and loss of experimental results

Training the selected Vision-Language-Action (VLA) models and their variants may require substantial computational resources and execution time. Training may be delayed by limited GPU availability, software or hardware errors, interrupted sessions, or unexpected termination before model checkpoints and experimental results are saved. Such incidents could delay subsequent activities, including evaluation, comparison, and report preparation.

To mitigate this risk, the team will use GPU-enabled Kaggle environments, subject to available computing resources and usage quotas. Both AI engineers will prepare their training environments and execute independent experiments in parallel where resource availability permits. Training configurations, logs, evaluation results, and model checkpoints will be saved regularly and copied to persistent storage.Training will be prioritized according to the project's experimental objectives, with sufficient time

reserved for evaluation and analysis. If the available compute budget becomes insufficient, the team will prioritize the essential baseline and primary experimental conditions before conducting supplementary experiments.

Risk 2: SO-ARM101 hardware failure

The project depends on the SO-ARM101 robotic platform for data collection and physical evaluation. Hardware failure caused by component wear, mechanical damage, electrical faults, or accidental mishandling could interrupt these activities. Since replacement components or a replacement device may require international shipping, equipment downtime could substantially delay the project.

To reduce this risk, the team will inspect and test the robotic platform before data collection and evaluation, follow appropriate operating procedures, and maintain backups of the collected datasets and associated configurations. A second SO-ARM101 arm will be prepared as a replacement, subject to successful assembly and compatibility testing. If the primary arm fails, the team will switch to the backup platform and verify its operation before resuming experiments. If both platforms become unavailable, the team will prioritize analysis of previously collected data, model training, software debugging, and other tasks that do not require physical access to the robot. Physical evaluation will resume when the equipment becomes available, and any resulting limitations will be documented in the final report.

Risk 3: Team member unavailability

A team member may become temporarily or permanently unavailable because of illness or other unforeseen circumstances. Since the project involves interdependent activities, including data collection, model training, evaluation, and report preparation, the absence of a member responsible for a critical task could delay dependent activities and create an uneven workload among the remaining members.

To mitigate this risk, the team will document task procedures, experimental configurations, data locations, and implementation details in a shared project repository. Each critical task will have a designated backup member who is familiar with its essential procedures. Team members will maintain sufficient cross-functional knowledge to take over urgent tasks when necessary. If a member becomes unavailable, the team leader will reassess task priorities, redistribute responsibilities according to each member's capacity and expertise, and adjust the schedule where necessary. Critical deliverables, particularly data collection, baseline experiments, evaluation, and report completion, will receive priority over nonessential extensions.

These risks will be monitored throughout the project. If a risk materializes or its likelihood increases, the team will assess its effect on the schedule and deliverables, implement the corresponding contingency plan, and document any resulting changes to the project plan. This approach is intended to reduce avoidable delays and preserve the feasibility of completing the project's essential objectives within the available time and resources.

2.3 Quality Management

[Details on the measures and processes that will be applied to ensure the quality of the project, including testing and validation of the product or project outcomes.]

2.4 Budget and Funding (Optional)

[If applicable, describe the project budget and funding sources. This should include details on estimated costs and financial management throughout the project.]

The project had no external funding for hardware. The team members self-funded the physical setup. Recorded spending from July to October 2026 was **12,504,000 VND** (about 3,126,000 VND per member), listed in Table X.

Table X. Hardware expenditure (thousand VND)

Category	Description	Cost
Servos	2 sets of 12 servos for SO-ARM101 (leader and follower arms), including visa/transfer fees. The second set supports the backup arm in Risk 2.	9,385
Control electronics	4 bus driver boards; 2 × 6V adapters	1,114
Cameras	3 camera modules and 1 additional camera	881
3D-printing material	3 rolls of PETG for the arm's shell	352
Shipping fee	China to Vietnam	105
Accessories	USB hubs, Type-C cables, tape, storage box, camera stand	667
Total		**12,504**

The partner lab sponsors the project by allowing the team to use its 3D-printing machine and PETG plastic. No money was spent on training: the team uses its own computers and the free Kaggle GPU quota, whose limits are covered in the risk plan.

2.5 Change Management Process

[Describe the process that will be followed for any changes in the project scope, schedule, resources, or other requirements.]

The change management process controls revisions to VISTA's scope, schedule, resources, and requirements while preserving the validity of its experimental comparisons. Changes to the SmolVLA backbone, dataset, training configuration, or evaluation protocol may affect both manipulation performance and computational efficiency. Consequently, revisions must be assessed and documented before their outcomes are incorporated into the final comparison. The process follows established change-control principles: recording requests, assessing impacts, obtaining approval, updating the baseline, and monitoring implementation [1].

Purpose and scope. Controlled changes include the backbone shortlist, task definitions, demonstration quantities, training steps, batch size, learning rate, evaluation conditions, personnel allocation, compute resources, budget, and arrangements for factory testing. Impact assessment will reference Protocol freeze (**12-Oct**), Review 2 (**27-Oct**), Dataset v1.0 frozen (**30-Oct**), Training complete (**20-Nov**), Review 3 (**TBC**), and Final submission (**30-Nov**). Changes after a relevant freeze require explicit versioning and consideration of whether previous experiments remain comparable. Formatting corrections, routine task coordination, and code refactoring that preserves computational behaviour do not require a separate change request unless they affect an approved baseline.

Integration with existing project management. The process operates through **WBS 1.4, "Risk & change-control log."** Nguyễn Gia Phát, the PM, is responsible for updating change records and affected plans within **one working day after agreement**. Google Drive holds reports, research documents, and the change-control record; ClickUp tracks assignments and deadlines; Kaggle holds training code. Discord supports communication, while Google Meet supports online meetings. Changes will be reviewed during full-team working sessions on Wednesday and Sunday and discussed during the 1.5-hour online meetings on other days when necessary. Updates will also appear in end-of-day reports and weekly stakeholder communications.

Step 1 — Change request. Any member may raise a request based on a technical issue, resource constraint, or revised requirement; stakeholder requests enter the same process. The input is a description of the problem and proposed adjustment. The requester provides the current and proposed states, reason, supporting evidence, affected work package or milestone, urgency, and acceptance criteria. Phát assigns a Change ID and records the request by the next working day. The output is a registered request awaiting assessment, rather than permission to implement it.

Step 2 — Impact assessment. The request and current baseline form the inputs. Phát coordinates an assessment covering scope, schedule, cost, quality, risk, dependencies, and external commitments. Nguyễn Minh Đức assesses backbone integration and training implications, reviews configuration

consistency and metric comparability; Nguyễn Duy Sơn and Nguyễn Lê Anh Quân assesses data collection, processing, and setup requirements. The assessment identifies additional work, possible retraining, effects on Kaggle GPU free quota availability, and expenditure against the **VND 12,504,000** budget. The output is a classification and recommended action, prepared before the next full-team review where practicable. Incomplete assessments remain pending.

Step 3 — Approval. The proposed approval structure distinguishes internal adjustments, research changes, and changes affecting Kahoku Lighting Solutions Vietnam Co., Ltd. The PM decides minor changes; medium changes require PM and supervisor agreement; major changes affecting factory commitments additionally require factory-partner agreement. Decisions will be recorded as *Approved*, *Rejected*, or *Deferred*, with the authority, date, reasons, and conditions. This structure is a prospective governance rule; historical records will identify the people who actually agreed rather than imply retrospective supervisor or partner approval.

Step 4 — Baseline update. Following approval, Phát updated the change record and affected Google Drive documents and ClickUp tasks within one working day. Relevant updates include the WBS, schedule, budget, risk register, dataset version, protocol version, and training configuration. Changes to Kaggle code will reference a saved notebook version and the Change ID. Affected experiment records will retain the dataset and configuration identifiers used for each run. The output is a traceable revised baseline, communicated through Discord and the existing reporting cycle before work proceeds under the revised conditions.

Step 5 — Implementation and closure. The assigned member implements the approved change and reports progress through existing meetings and task updates. Phát checks the evidence against the acceptance criteria at the next scheduled review after implementation. Outputs may include revised code, dataset checks, completed assignments, or updated testing arrangements. The request is closed only when implementation and consequential document updates are confirmed. Failed checks reopen the request or generate a follow-up request.

Table 5. Change Management Classification and Approval Authority

Level	Criteria	Examples in this project	Approval authority
Minor	Adjustments within approved scope, protocol, budget, and milestone dates.	Reallocating data-processing work to Quân and Sơn without changing team membership; rescheduling internal work; changing settings within an approved experimental range.	**PM**, following consultation with affected members.

Medium	Changes to research execution, dataset quantity, or resources that preserve core objectives and factory commitments.	Increasing demonstrations; reducing demonstrations from 300 to 250; changing training steps, batch size, or learning rate outside approved ranges; revising compute allocation; adding a spare servo, camera, or PETG expenditure.	**PM + supervisor**.
Major	Changes to core scope, frozen evaluation criteria, or external testing commitments.	Replacing LFM2.5-VL-0.45B or removing a backbone from the approved shortlist; changing the second task definition; revising evaluation after freeze; changing the task, setup, or agreed visit at Kahoku.	**PM + supervisor**; **factory partner additionally approves changes affecting factory commitments**.

Recorded project changes. The team also identified Phát as the decision-maker for increasing data, Quân and Sơn as additional data-processing support when progress is delayed, and Đức as using multiple Kaggle accounts for training. These records will retain their actual decision history. The data increase still requires a confirmed quantity and version; no unreported demonstration count or payload limit will be inferred. Training records will identify account, notebook version, configuration, and outputs to maintain traceability across runs.

Records, evaluation, and risk integration. Each change entry will contain Change ID, date, requester, description, reason, affected WBS item or milestone, classification, impact assessment, approval decision and authority, implementation status, and closure date. Supporting links will connect decisions to meeting notes, ClickUp tasks, and technical artifacts. Process compliance will require documented assessment and approval for controlled changes, updates within one working day, and closure evidence. Any increased exposure to training delay, SO-ARM101 failure, or team member unavailability will trigger a simultaneous risk-register update.

This process preserves experimental integrity by keeping changes to task definitions, datasets, backbone selection, and protocols visible and traceable throughout the final comparison.

2.6 Closure and Evaluation

[Describe the process for closing and evaluating the project upon completion, including how experiences and lessons learned from the project will be summarized.]

Project closure will formally confirm that VISTA's deliverables have been accepted, stakeholder obligations have been fulfilled, and research knowledge has been preserved. Completing the report alone will not constitute closure.

Deliverable acceptance. Phát will maintain a checklist against the approved scope. Nguyễn Minh Đức will provide backbone-integration code, training artifacts, and the app prototype. Nguyễn Lê Anh Quân will review technical consistency, evaluation outputs, and app tests. Nguyễn Duy Sơn will provide dataset records, setup documentation, and demonstration videos. The checklist will cover:

- The **Capstone Project Report** and two demonstration videos presenting the best variant and original SmolVLA, including failure cases.
- The **Control and Monitoring App prototype**, demonstrating policy execution and switching, live camera views, and 3D arm pose.
- The **reusable backbone-integration guide**, verified claim ledger, closed risk register and change log, and reconciled budget records.

The PM will confirm completeness, and the supervisor, Mr. Lưu Giang Nam, will be requested to confirm academic and technical acceptance. Kahoku Lighting Solutions Vietnam Co., Ltd. will be requested to confirm on-site testing and associated permissions where applicable. Verification will use **WBS 10.12, "Claim-ledger verification & quality gates (G1–G3)" (22-Nov–28-Nov)**. Its mandatory acceptance rule is that **"a different person verifies than the one who wrote it."** Records will identify the writer, verifier, evidence, and outcome. Material discrepancies must be resolved before **WBS 10.13, "Final proofreading & report freeze" (28-Nov–29-Nov)**.

Post-project evaluation. Actual outcomes will be assessed against objectives O1–O6 using quantitative and qualitative evidence. Measurements will include success rate with successful-trial counts and denominators, latency median and p95, memory usage, and parameter count. Comparisons will identify hardware, dataset versions, evaluation conditions, and protocol deviations. Qualitative assessment will consider demonstration quality, partner feedback where available, and independent use of the integration guide. Schedule and expenditure will also be compared with approved baselines. Verdicts remain blank until evidence review.

Objective	Evidence used	Success condition
O1: Design and implement alternative backbone architectures	Architecture documentation, GitHub code, interface checks, configuration files	LFM2.5-VL-0.45B and FastVLM-0.5B are integrated as alternatives to SmolVLM-2 within SmolVLA, with documented and executable interfaces.

O2: Train the baseline and modified models	Kaggle training logs, checkpoints, dataset identifiers, saved configurations	ACT, original SmolVLA, and both modified configurations complete the approved training protocol, with deviations recorded.
O3: Evaluate manipulation performance and computational efficiency	Trial records, success rate, latency median/p95, memory usage, parameter count	All four configurations receive comparable evaluation under the approved protocol, with uncertainty and limitations reported.
O4: Demonstrate real-world applicability	Two videos, app test records, hardware observations, partner feedback where available	Hardware demonstrations and app functions satisfy agreed acceptance checks, including documentation of failures and deployment limitations.
O5: Develop a reusable backbone-integration strategy	Integration guide, interface specifications, independent walkthrough	A member other than the guide's author can follow the integration workflow and identify backbone-specific adaptations.
O6: Assess the potential for improving new lightweight VLA models (highest-level)	Verified synthesis of O1–O5, performance–efficiency comparisons, limitations	Conclusions assess lightweight VLA potential within the tested conditions and distinguish demonstrated findings from future hypotheses.

Evaluation records will use "the second task" until its formal name is consistent throughout the report. Earlier references to "Halogen Lampshade Tray Packaging" must be reconciled with the team's agreed change to sorting and arranging lighting-related items. Amendments to a frozen definition will follow Section 2.5.

Lessons learned. The team will consolidate experiences during existing Wednesday and Sunday working sessions. Each lesson will record an observation, supporting evidence, root cause, consequence, and specific recommendation. These categories define the review process rather than assert that particular failures occurred.

Technical integration. Members will examine interface compatibility and layer handling for LFM2.5-VL-0.45B and FastVLM-0.5B. Where integration problems occurred, analysis will identify

dimensional assumptions, retained-layer choices, or interface mismatches; recommendations will specify assertions and integration checks.

Data. Collection, calibration, and episode quality will be reviewed. Where inconsistencies occurred, root causes will be traced to setup drift, collection instructions, or cleaning decisions. Recommendations will define calibration checks and episode-acceptance criteria.

Training and compute. The review will examine Kaggle GPU free quota constraints, notebook versions, checkpoint persistence, and recovery. Where interruptions caused lost work, members will identify the resource or saving failure and recommend checkpoint intervals, persistent storage, and restoration checks.

Management. Schedule estimates, risk responses, and change-control records will be compared with execution. Where delays or repeated work occurred, analysis will identify dependency errors or delayed decisions and recommend earlier escalation and revised scheduling allowances.

Teamwork and communication. Members will assess task ownership, data-processing support, and handovers. Where coordination problems occurred, the review will identify unclear responsibilities or missing information and recommend named owners and explicit handover requirements.

Deployment and safety. SO-ARM101 capability and factory arrangements will be reviewed. The reported lifting limitation behind the task change will inform a recommendation to assess object mass and handling feasibility before task commitment. Other recommendations will follow documented operating observations.

Archiving and handover. Training code will be executed on Kaggle and archived on GitHub for final submission. The handover package will include notebooks, configuration files, dependency information, and execution instructions. A final release tag will identify the submitted version. Google Drive will preserve dataset v1.0 with hashes, checkpoints, logs, videos, design documents, meeting minutes, change logs, risk records, and budget receipts. An appendix artifact index will list locations, versions, owners, hashes where applicable, and reproduction instructions. Another member will check accessibility and execute the documented evaluation workflow.

Closure sign-off. Phát will prepare a written closure summary containing acceptance evidence, objective assessments, lessons, artifact references, and obligation status. Supervisor sign-off and factory confirmation, where applicable, will be sought before **30-Nov**. This will authorise the final release. **WBS 10.14, "Final submission,"** remains scheduled for **30-Nov**; administrative closure will follow attachment of submission evidence and closure of **WBS 1.4**, including documented ownership of residual risks.

Project Success Criteria. Success will require timely accepted deliverables, evidence-based assessment of O1–O6, reproducible evaluation, independent claim verification, and stable app and hardware demonstrations against agreed checks. Technical improvement will require at least one variant to improve or demonstrate non-inferiority to original SmolVLA in success rate while meeting predefined computational-efficiency limits. Any non-inferiority margin must be established before outcome inspection. Negative or inconclusive findings will be reported accurately and distinguished from delivery completion.

Closure will require every reported result to trace to evidence, obligations to the supervisor, partner lab, and factory partner to be completed, and project knowledge to remain accessible for subsequent teams to reuse.

III. Existing Systems

[Provide the final existing systems following the template as part II in the Report #3]

[This Section is usually titled "Literature Review" or "Related Work". For projects related to specific technologies or systems, you might call this section "Existing Systems" or "State of the Art". Choose a title that fits the content and context of your project.]

1. Overview of the Field

[Provide an overview of the field of study related to your project. This helps place your project within the broader context of that field.]

Robotics aims to relieve people of repetitive, tiring, or hazardous physical work by producing motion that is useful, reliable, and safe. Since the field took shape in the 1950s, it has drawn on mechanical and electrical engineering, rigid-body mechanics, control theory, and software. Any working system must connect planning software to the hardware that carries out motion. **Fig. 1** shows the methods for motion generation, which are broadly categorized into two primary families:

+ *Explicit approaches* build on dynamics, using precise descriptions of a robot's rigid links and of their contact with the environment.
+ *Implicit approaches* treat motion as a statistical pattern to be learned from sensorimotor readings [9].

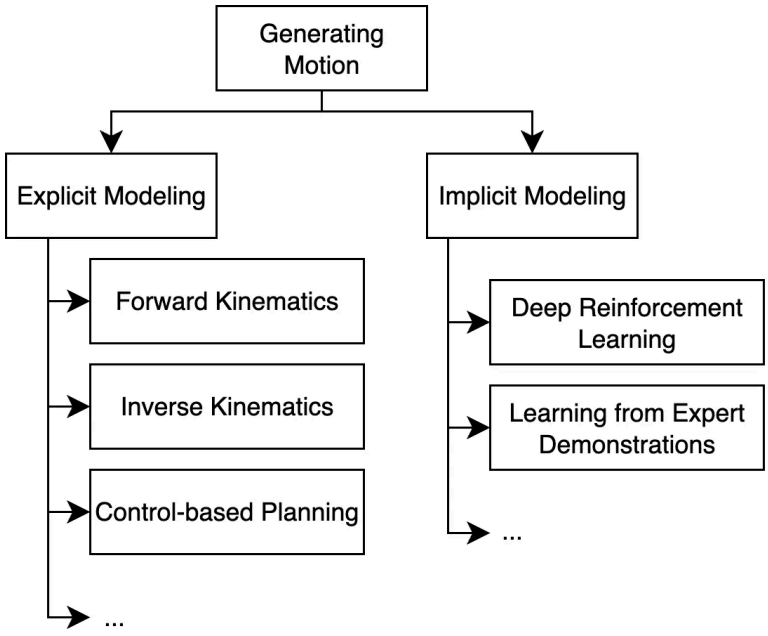


Figure 1. Taxonomy of Motion Generation Methods

Robot learning belongs to the second family. Its rise reflects how hard it is to derive accurate models of robot–environment interaction, and recent results in locomotion, whole-body control, and manipulation suggest that learned behavior can rival explicit modeling [10].

Learning-based methods offer several advantages over the classical stack. Visuomotor policies map sensor readings directly to actions, so the interfaces between separate perception, planning, and control modules disappear. Learned feature extraction readily absorbs different input modalities. Where dynamics are unknown or impractical to model, interaction data can replace an explicit model altogether. Robot learning can also draw on the growing supply of open robotics data, much as computer vision and natural language processing grew with large-scale corpora.

As illustrated in **Fig. 2**, two main classes of methods have gained prominence: Behavioral Cloning (BC) and Reinforcement Learning (RL). On physical hardware, RL is hard to apply because exploration is costly and potentially unsafe, dense rewards are brittle and task-specific, and episodic training needs dependable resets. BC avoids these obstacles by casting control as imitation of expert demonstrations. The learned behavior then inherits the goals and success criteria implicit in the data, and no hand-crafted reward is needed. This project lies within BC and covers its two main branches. Single-task policies, represented by Action Chunking with Transformers (*ACT*), are typically not language-conditioned and each performs one task. Generalist policies, represented by *SmolVLA*, are conditioned on language instructions and produce motion across many tasks. The two branches differ in spirit but share a training principle, since both reproduce trajectories from a set of demonstrations, and this common principle justifies treating them together. The application studied here is tabletop manipulation, in which a robot arm changes the state of objects in its workspace through dexterous motion.

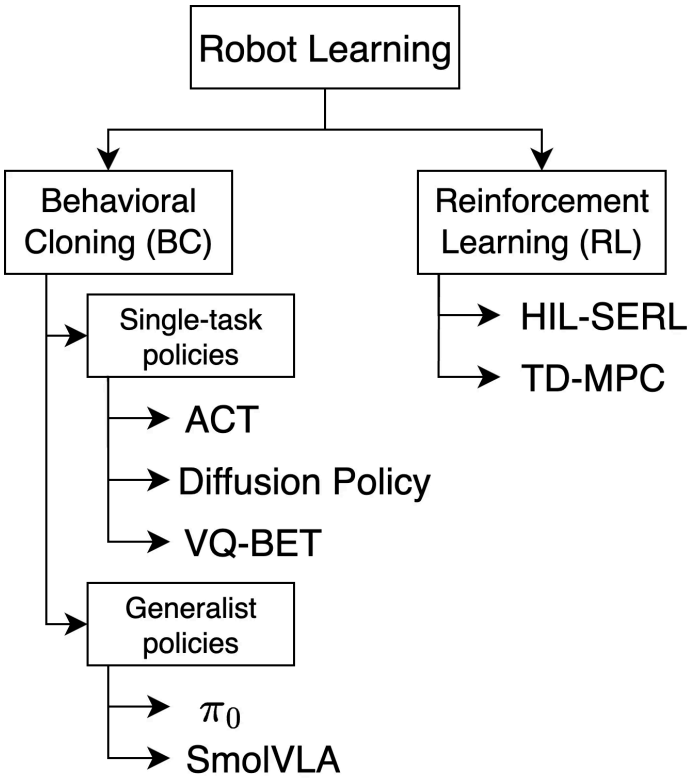


Figure 2. Taxonomy of Robot Learning Methods

2. Historical Context

[Provide some historical background or developments in the field, especially those directly related to your project.]

BC has a long history. Pomerleau [22] framed it as learning a mapping from observations to actions, a problem that can be approached through ordinary supervised learning. For decades afterward, robotics data stayed small and bound to individual tasks. Each expert trajectory was tied to one robot platform and the conditions of one environment, and human experts often performed the same task in quite different ways. Pooling such data was therefore difficult, and combining demonstrations from different embodiments could even cause negative transfer when control strategies conflicted. The result was a generation of specialist policies that performed well on their designated task but did not carry over to new deployments.

Two developments loosened these constraints. The first concerned hardware. ALOHA and ACT [34] showed that fine-grained manipulation could be learned on low-cost, imprecise arms, which encouraged community-built teleoperation systems, open datasets, and the LeRobot library (see ***Fig. 3*** for an overview of its end-to-end framework). The SO-ARM101 used in this project descends from that movement [6], [14].

The second concerned scale. Large-scale models in natural language processing and computer vision had shifted those fields from task-specific training to a two-stage pattern of broad pretraining followed by task-specific adaptation, and robotics researchers began to ask whether the same recipe could work for control. Early generalist efforts such as BC-Z [11], RT-1 [5], and RT-2 [36] were followed by open datasets, including Open X-Embodiment [20] and DROID [12], that made open-source models possible. OpenVLA [13], π0 [4], and SmolVLA [26] are the main examples (Kim et al., 2024; Black et al., 2024; Shukor et al., 2025). In parallel, decentralized collection by individual researchers and hobbyists, supported by LeRobot, has become a significant source of data. Generalist models have also grown smaller and easier to run on limited hardware, a trend that matters for robots deployed outside large laboratories.

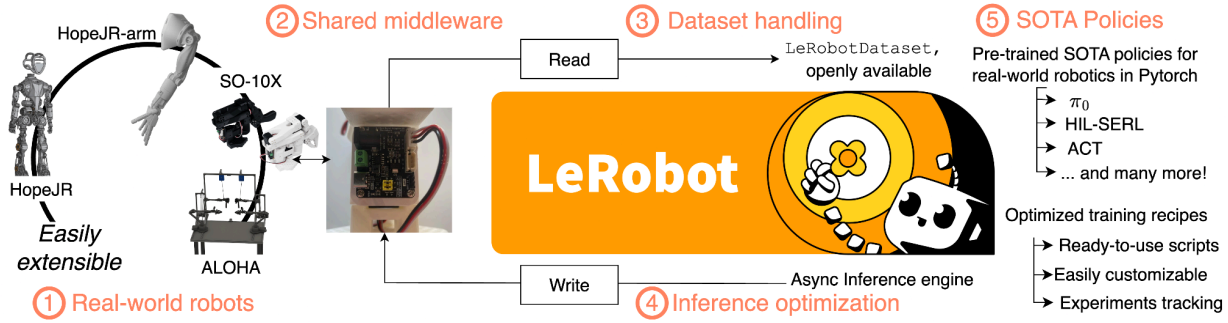


Fig 3. LeRobot (An Open-Source Library for End-to-End Robot Learning)

3. Key Studies and Theories

[Describe and assess key studies, theories, and published works related to your topic. This includes both fundamental and applied research.]

The development of robot learning has progressed from task-specific imitation-learning policies toward generalist systems capable of understanding visual observations, interpreting natural-language instructions, and generating corresponding robot actions. Early approaches primarily focused on learning motor behaviours from demonstrations, whereas more recent Vision-Language-Action (VLA) models incorporate pretrained vision-language models (VLMs) to transfer semantic knowledge into robotic control.

3.1. Task-Specific Imitation Learning: Action Chunking with Transformers (ACT).

Action Chunking with Transformers (ACT) [34] represents an important development in demonstration-based robot learning. Rather than predicting a single action at each time step, ACT employs a transformer-based policy to predict a sequence of future actions, referred to as an action chunk. Its architecture incorporates a Conditional Variational Autoencoder (CVAE) [27] to model variations in demonstrated action sequences, while temporal ensembling combines overlapping predictions during inference. Action chunking reduces the effective prediction horizon and helps mitigate error accumulation during sequential execution.

Unlike VLA models, ACT does not incorporate natural-language instructions or a pretrained vision-language backbone. Instead, it learns a task-specific mapping from visual observations and robot states to motor commands using collected demonstrations. Its relatively straightforward architecture and demonstration-based training paradigm make it a useful reference for evaluating the benefits of language-conditioned robotic policies. In this project, ACT therefore serves as a conventional imitation-learning baseline against which the performance of SmolVLA-based approaches can be assessed [34].

3.2. The Emergence of Vision-Language-Action Models: BC-Z, RT-1, and RT-2.

The introduction of language-conditioned robotic policies marked a transition from learning individual manipulation behaviours toward developing more generalizable robotic systems. BC-Z demonstrated that robot policies could be conditioned on task descriptions, enabling a single policy to perform multiple manipulation tasks. Building on this direction, RT-1 [5] expanded robotic learning through large-scale demonstrations collected across multiple robots and tasks. Its transformer-based architecture processes camera observations and natural-language instructions to predict discretized robot actions. This demonstrated the potential of scaling both demonstration datasets and policy architectures to improve task generalization [11].

RT-2 [36] further advanced this direction by introducing the Vision-Language-Action (VLA) paradigm. Rather than treating robotic control as an entirely separate prediction problem, RT-2 adapts pretrained vision-language models, including PaLI-X [7] and PaLM-E [10], to generate robot actions within the same token-prediction framework used for language generation. By combining web-scale visual-language knowledge with robotic demonstrations, the model can transfer semantic understanding to manipulation tasks involving objects and concepts that were not explicitly represented in its robotic training data. This approach illustrates the potential of pretrained multimodal representations to improve robotic generalization and establishes the VLM backbone as an important architectural component of VLA systems.

3.3. Large-Scale Robotic Datasets and Open VLA Architectures.

The development of generalist robotic policies has been closely associated with the availability of large and diverse demonstration datasets. Open X-Embodiment consolidated robotic datasets from multiple institutions and embodiments, providing approximately 1.4 million trajectories for studying cross-robot learning and transfer. Its findings demonstrated the potential advantages of training policies on diverse multi-embodiment data rather than relying exclusively on demonstrations collected from individual robotic platforms. Similarly, DROID introduced a large-scale dataset of real-world robotic interactions, further expanding the availability of demonstrations collected in diverse environments [20], [12].

Despite these advances, access to large proprietary VLA systems and their training resources remains a barrier to independent research. OpenVLA addressed part of this limitation by releasing model weights, training procedures, and evaluation resources. It combines a pretrained vision encoder with a Llama 2 7B [28] language model, projecting visual representations into the language model's embedding space before generating discretized robot actions. Although this architecture makes large-scale VLA research more accessible, its parameter count and computational requirements still present challenges for researchers working with limited hardware resources [13].

3.4. Lightweight Vision-Language-Action Models: SmolVLA.

SmolVLA [26] addresses the computational accessibility challenges associated with large-scale VLA architectures. It introduces a compact and efficient framework designed to support robotic learning and deployment on more affordable hardware. The architecture combines a pretrained SmolVLM vision-language backbone with a dedicated action expert that generates continuous robot actions using flow matching. Several architectural optimizations, including visual token reduction, selective vision-model layer execution, and interleaved attention mechanisms, are employed to reduce computational overhead while retaining the benefits of pretrained multimodal representations [17].

Besides, SmolVLA introduces an asynchronous inference framework that separates action prediction from robot execution, improving responsiveness during real-world deployment. Its compatibility with affordable robotic platforms, including the SO-100 and SO-101, makes it particularly relevant to research conducted with low-cost manipulators. Nevertheless, reducing model size does not necessarily eliminate limitations associated with the representational capabilities of the underlying VLM backbone. The backbone determines how visual information and language instructions are encoded before being incorporated into the action-generation process. Consequently, its architecture and pretrained capabilities may influence the model's ability to distinguish objects, interpret spatial relationships, and understand task instructions [26].

3.5. SmolVLM2 - Lightweight Vision-Language Backbone.

SmolVLM2 [19] is a family of compact vision-language models developed to provide multimodal understanding capabilities with relatively low computational requirements. It supports visual and textual reasoning tasks, including image description, visual question answering, and document understanding. Its lightweight architecture makes it suitable for integration into resource-constrained applications, including robotic systems. However, most of it training data is for document reading and OCR tasks,

which leads to it weakness in handle images. This raises the possibility of replacing SmolVLM2 in SmolVLA with alternative lightweight VLMs that may offer more suitable representations for robotic control while maintaining comparable computational efficiency [26].

This consideration motivates the present project to investigate alternative lightweight VLM backbones within the SmolVLA framework. Rather than developing an entirely new VLA architecture, the project examines whether replacing the original visual-language backbone with alternative compact multimodal models can improve task performance while maintaining computational efficiency on the SO-ARM101 platform.

4. Technological Advancements

[Discuss recent or significant technological advancements in your field, and how they affect your project.]

Three developments have made this project feasible on a student budget. Low-cost open hardware lowered the price of collecting robot data. An open software stack unified how that data is stored, used for training, and deployed. Compact vision-language models (VLMs) then reached a size where they can serve as backbones for lightweight policies. Each affects a different part of the project, from data collection to the design of the model.

Low-cost, open robot hardware. Early learning-based manipulation needed expensive, precisely engineered arms. ALOHA showed that fine-grained tasks could be learned on inexpensive teleoperated hardware, and ALOHA 2 [1] later refined this approach. The SO-100 and SO-101 arms [14] extend it further. They use consumer-grade parts and 3D-printable shells, and the LeRobot authors put a single arm at roughly 225 euros, against about 21 thousand euros for the ALOHA bimanual platform. The arms come in leader–follower pairs. A person moves the leader arm, the follower copies the motion, and the recorded motion becomes training data. The same pair of arms therefore serves for both demonstration and deployment. This is the setup used in the project, and it is why a student team can collect its own task-specific demonstrations instead of relying only on public datasets [6].

1. An open software and data ecosystem: LeRobot. Hardware alone leaves a practical gap, because robot data has historically been scattered across incompatible formats and robot-specific control code. LeRobot [6] addresses this with a single library covering the whole stack. A shared Python middleware reads the leader arm and drives the follower, whether under teleoperation or under a learned policy. The LeRobotDataset format stores sensor readings, multi-camera video, and task descriptions in one schema and can be streamed without downloading the full corpus. By September 2025, over 16 thousand datasets from more than 2.2 thousand contributors had been shared in this format, and over half of those datasets came from SO-10X arms. The library also provides tested PyTorch implementations of ACT, Diffusion Policy, π0, and SmolVLA, with training scripts for each. Its inference stack separates action prediction from action execution, so a policy can run on a separate, more powerful machine and stream action chunks to the robot. In the authors' real-world tests on the SO-100, this asynchronous scheme gave similar success rates to synchronous inference but shorter cycle times, although the test used only ten episodes per task. For this project, LeRobot is the common platform on which all four model configurations will be trained, run, and compared under identical conditions [8].

2. Generative action models and compact VLAs. Modern policies no longer predict a single action per step. ACT predicts short chunks of future actions with a conditional variational autoencoder. Diffusion Policy and later flow-matching models such as π0 and SmolVLA learn the full distribution of actions, which suits demonstration data where experts act differently from one attempt to the next. Within this line, model size has become a design variable. LeRobot's measurements place ACT at about 52 million parameters and SmolVLA at about 450 million, compared with 3.5 billion for π0. In fp32, SmolVLA peaks at roughly 1.75 GB of GPU memory, while π0 needs about 13 GB. This makes a language-conditioned policy practical on a single consumer GPU and gives SmolVLA a clear role as the efficiency-oriented generalist baseline here [34], [8], [4], [26], [6], [17] .

3. Compact VLMs as backbones. The most recent change is in the VLMs that supply a VLA's visual and linguistic understanding. According to Li et al. [16], VLM design has moved from separate vision and language towers, through projector-based adaptation of a pretrained language model, to natively multimodal trunks. Model scale has diverged along the way. Frontier systems have grown toward trillions of parameters, while a parallel line has shrunk to sub-billion models for edge deployment. Two members of this second line are central to this project. FastVLM [29] pairs a hybrid convolution–transformer encoder, FastViTHD, with a small language model and controls the visual token count through input resolution alone, avoiding a separate pruning step. LFM2-VL, available at 450 million parameters, builds on a hybrid backbone of convolution and attention layers Both target low latency and a small memory footprint, the same constraints SmolVLA was designed around [18], [2] .

These developments set the project's boundaries. Open arms and LeRobot make it possible to collect data, train, and deploy within the team's own resources. Flow-matching VLAs with small action experts leave room to change the backbone without redesigning the policy. The newer compact VLMs offer candidate replacements that differ from SmolVLM in pretraining data and in how they handle visual tokens. Their published results come mostly from image-question answering, document, and chart benchmarks, and FastVLM's reported latencies were measured on Apple silicon. Whether these gains carry over to manipulation success and to inference speed on the project's hardware has not been established, which is the question the experiments are designed to answer. The LeRobot authors also acknowledge that quantization and graph compilation, which would further cut inference cost, are outside the library's current scope, so the efficiency figures measured here should be read as unoptimized baselines [6], [29].

5. Comparison of Existing Systems

[If your project involves systems or technology, compare and contrast existing systems or technologies, including their advantages and disadvantages compared to your project.]

This section presents two comparisons to establish the motivation behind the proposed project. The first examines existing robot control policies, focusing on their architectures, capabilities, and computational requirements. The second compares robotic hardware platforms to justify the selection of SO-ARM101 as an affordable and accessible platform for VLA research. Together, these comparisons highlight the need for improving the capabilities of lightweight VLA models while maintaining compatibility with low-cost robotic hardware.

5.1. Comparison of Existing Robot Control Policies

Recent advances in robot learning have introduced different approaches to learning manipulation policies from demonstrations and multimodal observations. Among these approaches, Action Chunking with Transformers (ACT), π0, and SmolVLA represent different trade-offs between computational efficiency, task understanding, and general-purpose manipulation capabilities.

Aspect	**ACT [34]**	**π0 (Pi-Zero) [4]**	**SmolVLA [26]**
Model category	Imitation learning policy	Generalist VLA	Lightweight VLA
Core architecture	Conditional VAE and Transformer	Pretrained VLM and flow-matching action expert	Pretrained VLM and flow-matching action expert
Language conditioning	Not inherently supported	Supported	Supported
Visual understanding	Learned visual features using CNN backbone	Pretrained vision-language representations	Pretrained vision-language representations
Action generation	Chunked action prediction	Continuous action generation through flow matching	Continuous action generation through flow matching
Model parameters	Approximately 80M	Approximately 3B	Approximately 450M
Main advantages	Computationally efficient, relatively easy to train, effective for fine-grained imitation learning	Strong general-purpose manipulation capabilities, multimodal understanding, and cross-task potential	Smaller architecture, open-source implementation, and reduced computational requirements
Main limitations	Limited language-based task understanding and dependence on task-specific demonstrations	High computational and data requirements compared with compact policies	Reduced model capacity and potential limitations inherited from its pretrained VLM backbone

Table … . Policy comparisons

ACT provides an efficient baseline for learning task-specific manipulation from demonstrations, but does not inherently offer the language-conditioned semantic understanding of VLA models. In contrast, π0 demonstrates the potential of large-scale vision-language-action architectures, although its computational requirements may limit accessibility for resource-constrained research environments. SmolVLA occupies an intermediate position by providing multimodal task understanding with a considerably smaller architecture. Nevertheless, the efficiency of SmolVLA does not necessarily guarantee optimal visual and semantic representations for every manipulation task. Since its action-generation component depends on representations produced by the pretrained VLM backbone, the choice of backbone may influence downstream policy performance [34], [4], [26] .

Therefore, this project investigates whether replacing the original VLM backbone with an alternative lightweight vision-language model can improve the representations available for robotic control while maintaining reasonable computational requirements. This approach aims to investigate the potential to improve SmolVLA without introducing the computational overhead associated with substantially larger VLA architectures.

5.2. Comparison of Existing Robotic Hardware Platforms

The choice of robotic hardware is another important consideration in developing and evaluating VLA systems. Different robotic platforms provide different levels of mechanical capability, precision, accessibility, and financial requirements. This project considers Franka Emika Panda, ALOHA, and SO-ARM101 as representative platforms with different hardware designs and research objectives.

Aspect	**Franka Emika Panda**	**ALOHA [34]**	**SO-ARM101 [14]**
Platform type	Single-arm research manipulator	Bimanual teleoperation system	Low-cost leader-follower robotic arm system
Configuration	Single arm	Two robotic arms	Leader and follower arms
Degrees of freedom	7	6 per arm in the original setup	5 + gripper per arm

Actuation	High-performance robotic actuators	Dynamixel-based actuators	Feetech bus servos
Main strengths	High precision, reliability, and advanced manipulation capabilities	Bimanual manipulation, teleoperation, and fine-grained demonstration collection	Low hardware cost, open-source design, and accessible teleoperation
Main limitations	Higher acquisition cost and hardware requirements	More complex setup and higher overall hardware cost than a single low-cost arm	Limited payload, mechanical rigidity, and precision compared with research-grade manipulators
Primary research suitability	Advanced manipulation and robotic control research	Bimanual imitation learning and complex manipulation	Accessible imitation learning and VLA experimentation
Image			

Table … . Hardware Comparison

Franka Emika Panda. The Franka Emika Panda is a seven-degree-of-freedom collaborative robotic arm designed for research and industrial applications. Its advanced actuation, reliable control interfaces, and high-precision manipulation capabilities make it suitable for investigating complex robotic tasks. However, its acquisition and maintenance costs are considerably higher than those of low-cost educational robotic platforms. This creates a financial and infrastructure barrier for smaller research teams and institutions seeking to conduct repeated real-world manipulation experiments.

ALOHA. ALOHA, introduced by Zhao et al. , is a low-cost, open-source bimanual teleoperation system designed for collecting demonstrations and learning fine-grained manipulation skills. It combines two robotic arms with a leader-follower teleoperation interface, allowing human operators to demonstrate coordinated manipulation tasks. The original system demonstrated the ability to perform challenging tasks, including cable-tie threading, object insertion, and other contact-rich manipulation activities. Although ALOHA significantly improves accessibility compared with conventional high-end robotic

setups, its bimanual configuration requires more hardware components, space, and setup effort than a single-arm platform. Furthermore, its original hardware configuration was developed around higher-cost robotic components than those used in SO-ARM101 [34].

SO-ARM101. SO-ARM101 is an open-source, low-cost robotic arm platform developed within the SO-ARM project and integrated with the Hugging Face LeRobot ecosystem [14]. [6]. Its leader-follower configuration allows users to collect demonstrations through direct teleoperation and subsequently use the collected data to train imitation-learning and VLA policies. The platform uses affordable bus servos and 3D-printed structural components, substantially reducing the hardware requirements compared with conventional research manipulators. Despite its limitations in payload capacity, mechanical rigidity, and positioning precision, SO-ARM101 provides sufficient capabilities for investigating fundamental tabletop manipulation tasks, including object picking, placing, and stacking. Its compatibility with existing machine-learning tools and open-source hardware design also makes it easier to reproduce experiments and modify the system for different research requirements.

These platforms demonstrate that meaningful robotic manipulation research does not necessarily require expensive, high-performance industrial hardware. Although Franka Panda provides advanced mechanical capabilities and ALOHA enables complex bimanual manipulation, their hardware configurations introduce additional financial and infrastructure requirements. SO-ARM101 offers a more accessible alternative for studying imitation learning and VLA-based control, particularly for tabletop manipulation tasks that do not require high payload capacity or advanced force control. Importantly, the purpose of selecting SO-ARM101 is not to demonstrate that it can replace more advanced research manipulators in every application. Instead, it provides a controlled and affordable environment for investigating whether lightweight VLA architectures can achieve reliable manipulation performance under practical hardware constraints.

By combining a low-cost robotic platform with an improved lightweight VLA policy, this project aims to contribute toward making VLA experimentation more accessible to researchers and educational institutions with limited resources.

6. Gaps in the Literature/Technology

[Identify gaps or limitations in the current research or technological systems, and how your project intends to address or improve upon these issues.]

This section identifies the research gaps that motivate the proposed improvements to SmolVLA. Rather than developing an entirely new Vision-Language-Action (VLA) architecture, this project investigates whether the limitations of an existing lightweight architecture can be addressed through targeted modifications. Two main limitations are considered: the trade-off between model capability and computational efficiency, and the suitability of the original Vision-Language Model (VLM) backbone for robotic manipulation.

Gap 1: The trade-off between computational efficiency and model capability

One of the fundamental challenges in developing lightweight VLA systems is balancing computational efficiency with manipulation performance. In its discussion of limitations and future directions, the SmolVLA paper acknowledges that its compact architecture leaves room for further scaling and performance improvements. However, increasing model capacity may introduce additional computational requirements, potentially compromising the efficiency and accessibility that distinguish SmolVLA from larger VLA systems [26].

This challenge is also reflected in the development of subsequent VLA architectures. As presented in Table X, a comparison of VLA models introduced after SmolVLA's release in June 2025 illustrates the differences in model parameter counts across more recent approaches. Although newer architectures introduce improvements in various aspects of robotic control and multimodal understanding, these developments do not necessarily preserve the compact computational footprint required for resource-constrained environments.

Based on the models examined in our literature review, SmolVLA, with approximately 456 million parameters, remains a particularly lightweight VLA architecture with competitive manipulation capabilities. However, the available evidence does not establish whether its performance–efficiency balance can be improved further without substantially increasing its model size. This leaves an important question: whether the existing architecture can achieve better manipulation performance through targeted modifications rather than simply increasing its overall parameter count [26].

Policy	**No.Parameters in Billions**	**Paper**	**Published date**
S^2-VLA [30]	2B	*S^2-VLA: State-Space Guided Vision-Language-Action Models for Long-Horizon Manipulation [18]*	2026-06-29 (arXiv:2606.27872)
FabriVLA [31]	0.88B	*FabriVLA: A Lightweight Vision-Language-Action Model with Conformal Action Chunk Uncertainty*	2026-07-10 (arXiv:2607.08575)
PredVLA [24]	~0.68M	*PredVLA: A Sub-Million-Parameter Predictive-Coding Policy for Robot Manipulation*	2026-08-27 (arXiv:2608.26673)
MINERVA [25]	0.54M	*MINERVA: How Small Can a Manipulation Policy Be and Still Solve LIBERO?*	2026-09-03 (arXiv:2609.03715)
V-JEPA Policy [32]	0.9B	*V-JEPA Policy: Building Effective World-Action Models on Predictive Visual Latents*	2026-09-29 (arXiv:2609.37250)

SPAR [35]	1.12B	*SPAR: Structural Pruning and Action Recovery for Vision-Language-Action Models*	2026-10-19 (arXiv:2610.26487)
RynnVLA-Latent [33]	4B	*RynnVLA-Latent: Scaling Robotic Manipulation Foundation Models with 3D-Grounded Latent Actions*	2026-10-25 (arXiv:2610.34356)

Table … . VLA model's parameter comparison

Gap 2: Limitations of the original VLM backbone for robotic manipulation

A second limitation explicitly acknowledged by the SmolVLA authors concerns their choice of vision-language backbone. SmolVLA adopts SmolVLM-2, an off-the-shelf lightweight VLM, as its visual-language component. Although this choice contributes to the model's computational efficiency, its original pretraining distribution may not be optimally aligned with the visual understanding requirements of robotic manipulation. In particular, the SmolVLA paper identifies the backbone's emphasis on document understanding and optical character recognition (OCR) as a potential limitation when handling visually demanding manipulation tasks, such as object identification and spatial localization [26].

To investigate this limitation further, we examined the original SmolVLM paper, *"SmolVLM: Redefining Small and Efficient Multimodal Models,"* rather than relying exclusively on the observations reported by the SmolVLA authors. Its visual pretraining pipeline provides additional evidence regarding the distribution of training data [19].

For static image pretraining, approximately 48% of the image training mixture is allocated to OCR and document-related tasks. The remaining data covers general image captioning (14%), reasoning and logic (9%), table understanding (9%), chart understanding (12%), and visual question answering (8%). Meanwhile, the video fine-tuning mixture emphasizes information-seeking and visual-description tasks, with information-seeking data accounting for approximately 60% and visual description and captioning accounting for approximately 76% of the video data. Temporal understanding represents approximately 18% of this mixture [19].

These training distributions suggest that SmolVLM was developed primarily as a compact general-purpose multimodal model rather than specifically for the spatial reasoning and visual grounding requirements of robotic manipulation. Although its existing capabilities are sufficient to support SmolVLA's original design, its pretraining characteristics raise a concern about whether alternative lightweight VLMs could provide more suitable visual representations for downstream manipulation tasks.

Proposed research direction

The two limitations above motivate an investigation into improving SmolVLA without replacing its entire architecture. Instead of developing a new VLA policy from scratch or increasing the model's overall capacity indiscriminately, this project proposes retaining SmolVLA's existing architecture while replacing its original VLM backbone with carefully selected alternatives.

The replacement backbones will be selected according to several criteria, including parameter count, higher evaluation metrics, and suitability for integration into the existing SmolVLA architecture. In particular, the selected models should offer competitive visual capabilities while remaining sufficiently lightweight to preserve the computational advantages of the original policy.

However, replacing the backbone is not necessarily a straightforward process. Different VLMs may employ different visual encoders, language-model architectures, hidden dimensions, token representations, and layer configurations. Consequently, the original SmolVLA components cannot necessarily be reused without modification. This introduces an additional technical challenge: developing an adapter and integration mechanism that allows alternative VLM backbones to communicate effectively with SmolVLA's existing action expert.

Accordingly, this project investigates the following research question: **Can a carefully selected alternative VLM backbone improve manipulation performance while preserving, or further improving, SmolVLA's computational efficiency?**

To investigate this question, the project will integrate LFM2.5-VL-0.45B and FastVLM-0.5B as alternative backbones, producing modified SmolVLA variants for training and evaluation. Their performance will be compared against the original SmolVLA and ACT baselines on the SO-ARM101 robotic platform. Both manipulation performance and computational efficiency will be evaluated to determine whether backbone replacement provides measurable improvements without compromising the lightweight characteristics of the original architecture.

7. Justification for the Project

[Based on the above information, highlight the necessity and uniqueness of your project. Explain why your approach to the problem is important and how it differs from or improves upon existing research or systems.]

Necessity of the Project. The development of SmolVLA addresses an important challenge in Vision-Language-Action (VLA) research: making general-purpose robotic policies more accessible by reducing their computational requirements. Unlike many earlier VLA systems that rely on large-scale models, SmolVLA is designed as a compact and computationally efficient policy that can be trained and deployed using a single GPU. This design objective is particularly relevant to individual researchers, students, and institutions that do not have access to high-performance computing infrastructure. For comparison, OpenVLA contains approximately 7 billion parameters, while π_0 contains approximately 3.5 billion parameters. Although these models demonstrate the potential of large-scale VLA architectures, their substantial computational and memory requirements can create practical barriers to local experimentation, particularly on consumer-grade computers with limited GPU memory, such as those

equipped with 8 GB of VRAM or less. Consequently, computational accessibility remains an important consideration in the development and adoption of VLA technologies [26], [13], [4].

Nevertheless, the development of newer VLA architectures continues to introduce increasingly sophisticated models, mechanisms, and training approaches, as summarized in Table X. While these developments may offer improvements in general capabilities and task performance, they do not necessarily address the computational accessibility challenge. In particular, the models examined in this study generally retain substantially larger parameter counts than SmolVLA. This observation highlights a continuing gap between improving the capabilities of VLA systems and making those capabilities accessible under limited computational resources. Therefore, further investigation into lightweight VLA architectures remains necessary, not only to reduce hardware requirements but also to enable broader experimentation, reproducibility, and practical adoption within the research community.

Originality of the Project. Beyond the motivation to preserve computational efficiency, this project investigates whether the performance of a lightweight VLA policy can be improved through a more systematic selection and integration of its Vision-Language Model (VLM) backbone. SmolVLA employs SmolVLM-2 as its original visual-language backbone. However, the suitability of this backbone for robotic manipulation tasks should not be assumed to be optimal simply because it provides an efficient foundation. Different VLMs exhibit different capabilities, architectural characteristics, and computational requirements. Consequently, selecting an alternative backbone may offer opportunities to improve visual understanding and task-relevant representations while retaining the efficiency of the overall policy [26].

To investigate this possibility, the proposed project extends beyond conventional backbone replacement by developing an adapter mechanism to facilitate the integration of alternative VLMs into the SmolVLA architecture. Since candidate VLMs may differ in feature dimensions, visual token representations, encoder structures, and intermediate feature outputs, direct substitution is not necessarily feasible. The adapter is therefore intended to provide a more systematic integration mechanism, reducing the need to develop an entirely separate, model-specific integration procedure for every candidate backbone. Its effectiveness and compatibility will be examined through the integration and evaluation of selected lightweight VLMs.

Furthermore, this project aims to establish an experimental framework for selecting suitable VLM backbones for lightweight VLA policies. Rather than evaluating candidate models solely according to their standalone visual-language capabilities or parameter counts, the selection process will consider their suitability for robotic manipulation, compatibility with the existing policy architecture, and computational requirements. The resulting configurations will be evaluated against the original SmolVLA and ACT baselines on the SO-ARM101 platform. This evaluation is intended to determine whether alternative backbone configurations can improve manipulation performance without introducing an unacceptable increase in computational cost.

Overall, the project's contribution lies in investigating a systematic approach to improving lightweight VLA policies through backbone selection and integration. By combining an adapter mechanism with an evaluation framework, the project seeks to examine whether improvements in task performance can be achieved while preserving the computational accessibility that motivates the design of SmolVLA.

[Note: This section is not just a list of studies but an analysis and evaluation part where you connect existing systems/research and works to your project, showing their linkage and impact on your work.]

IV. Methodology

[Provide the final methodology following the template as part II in the Report #4]

[This section will detail how you will approach the project, with a focus on applying and implementing AI techniques. The team is allowed to customize the content of this section, however, the team needs to provide a detailed view of the methods and techniques used. To ensure that the reader understands the scientific and systematic approach that you apply to your capstone project in the field of AI. Below are two sample templates for this section that you can choose from or customize.]

IV. Methodology (sample 1)

1. Research Questions and Objectives

[Identify the specific research questions and objectives that the project will address through the application of machine learning.]

SmolVLA.

2. Environment Setup

[Describe the environment in which the reinforcement learning agent will operate, including parameters, states, and rewards.]

As discussed in the preceding sections, this project aims to evaluate the developed policies through two real-world manipulation tasks: *Desk Stationery Decluttering*, which serves as the pilot task, and [], which serves as the secondary task. Although each task requires a task-specific policy configuration, both are based on tabletop manipulation and therefore share several common environmental characteristics. The primary differences lie in the objects being manipulated, their initial and target positions, and the corresponding actions required from the SO-ARM101 robotic arm.

This section describes the environmental setup through three main components. First, Hardware Preparation introduces the devices and equipment used in the experiments, including their configurations and the rationale behind their selection within the LeRobot framework. Second, Desk Stationery Decluttering Setup describes the pilot task, its environment, the objects involved, and the manipulation actions required, together with the practical motivation for selecting this task. Third, … Setup presents the corresponding environment, objects, and manipulation procedures for the secondary task, explaining its intended real-world application.

a) Hardware setup

Development of Low-Cost Robotic Manipulation Platforms. Advances in open-source hardware, 3D printing, and accessible electronic components have made robotic manipulation platforms increasingly attainable for research and educational purposes. Traditionally, experiments involving real-world robotic manipulation have often relied on high-end robotic platforms, which can introduce considerable financial and technical barriers for individual researchers and small research teams. The development of

more affordable and reproducible robotic systems has therefore become an important direction for expanding access to robot-learning research.

An important example of this development is ALOHA, introduced by Zhao et al. in *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware*. ALOHA, see in Fig. , demonstrates a low-cost, open-source hardware system for bimanual teleoperation, with a reported total hardware cost of less than USD 20,000 using off-the-shelf robotic arms and 3D-printed components. The system adopts a leader–follower configuration in which human-operated leader arms provide demonstrations while follower arms reproduce the corresponding movements. This arrangement enables the collection of real-world manipulation demonstrations for imitation learning without requiring manually programmed trajectories.

Following this broader effort to improve hardware accessibility, TheRobotStudio, in collaboration with Hugging Face, introduced the SO-ARM100 as an open-source robotic arm platform. Its subsequent version, SO-ARM101, builds upon the original design with improvements to wiring, assembly procedures, and leader-arm motors. The platform is designed to be assembled from commercially available components and 3D-printed parts, allowing users to construct, customize, and adapt the hardware according to their experimental requirements. Its integration with Hugging Face's LeRobot framework further supports its use in robot-learning workflows, including teleoperation, demonstration collection, and policy deployment.

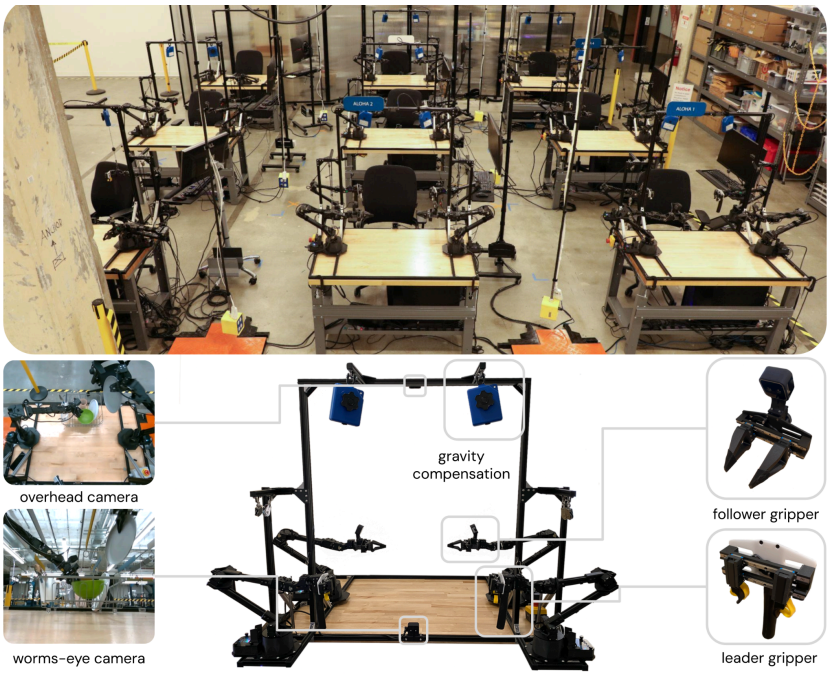


Fig . ALOHA Bimanual Robotic Arms

SO-ARM101 Robotic Arm. SO-ARM101 is a compact, open-source robotic arm intended for accessible robotic manipulation and robot-learning research. Each arm has five articulated joints and one gripper, commonly described as five degrees of freedom plus a gripper. Its mechanical structure is designed to

support tabletop manipulation while remaining relatively compact and portable. The open-source design also allows users to reproduce the hardware, replace components, and modify the configuration to accommodate different experimental requirements.

In this project, the original SO-ARM101 mechanical design is retained, while the servo configuration is adapted using alternative components that are locally available in Vietnam. This modification is intended to make hardware sourcing and maintenance more practical for the research team. As the resulting configuration differs from the standard reference design, the exact servo models, gear ratios, and other relevant specifications used in the constructed arms are documented in the following hardware tables.

The compact form factor and accessible construction of SO-ARM101 make it suitable for the tabletop manipulation experiments investigated in this project. In particular, the platform provides a physical environment for collecting demonstrations, training imitation-learning policies, and evaluating the performance of Vision-Language-Action models under real-world operating conditions.

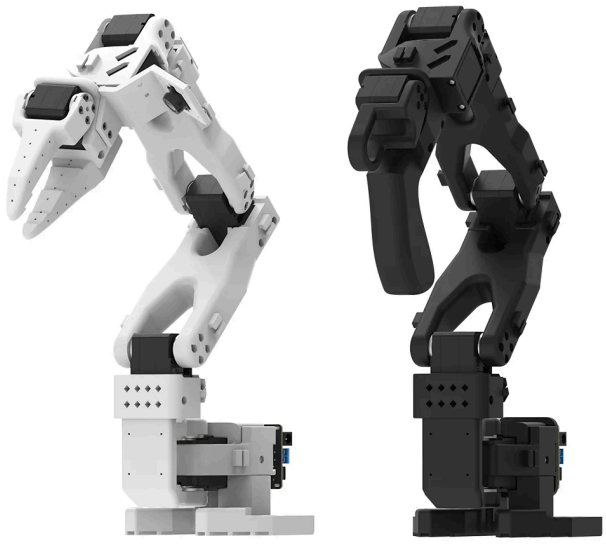


Fig . A set leader-follower of So-arm101

Leader–Follower Configuration. The SO-ARM101 teleoperation setup, shown in Fig., consists of two separate robotic arms: a **leader arm** and a **follower arm**. Although they share a similar mechanical structure, their functions and motor configurations differ according to their respective roles in the robot-learning workflow.

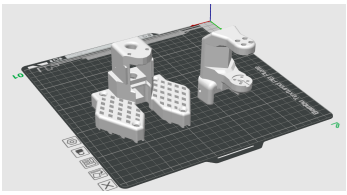
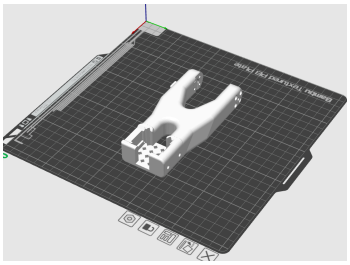
The leader arm serves as the human-operated input device. During demonstration collection, a human operator manually moves the leader arm to perform the target manipulation task. The corresponding

joint movements are transmitted to the follower arm through the teleoperation mechanism, allowing the follower to reproduce the demonstrated movements in real time. This process provides an intuitive approach to collecting robot demonstrations without requiring the operator to manually specify individual joint commands or program the complete motion trajectory.

The follower arm serves as the execution component of the setup. During teleoperation, it reproduces the movements of the leader arm, enabling the collection of corresponding robot observations and actions. These recorded trajectories can subsequently be organized into a dataset for training imitation-learning policies through *Behaviour Cloning (BC)*. During policy deployment and real-world evaluation, the trained model directly controls the follower arm to execute the learned manipulation tasks. Therefore, the leader arm is primarily required during demonstration collection, whereas the follower arm is used for both demonstration execution and autonomous policy evaluation.

The two tables, Table … and …, below summarize the hardware configurations of the leader and follower arms constructed for this project. They document the individual components and their corresponding servo specifications, providing a reference for understanding the differences between the two configurations and reproducing the experimental hardware setup. As for the shell of the arm we do 3D printing with PETG plastic, a hard and long-standing plastic, for long-term usage.

Table . Leader hardware specifications

Arm Axis	**Gear Ratio**	**Joint Limits (Degrees)**	**Total Range**	**Link Distance**	**Description**	**Visual Overview**
Base / Shoulder Pan	1:191	−110.00° to +110.00°	220.00°	73.5 mm	Rotates the entire arm assembly horizontally around the Z-axis.	
Shoulder Lift	1:345	−100.00° to +100.00°	200.00°	64.7 mm	Pitch axis joint located at the shoulder. Handles the primary vertical lifting load against gravity.	

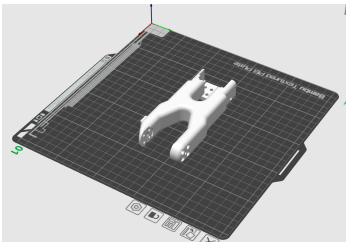
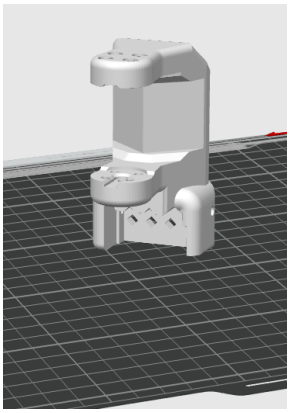
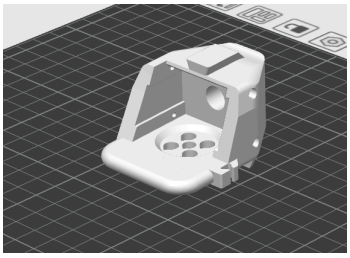
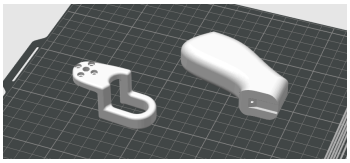
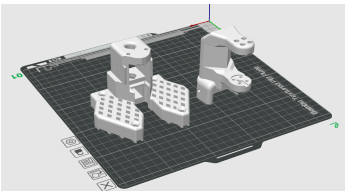
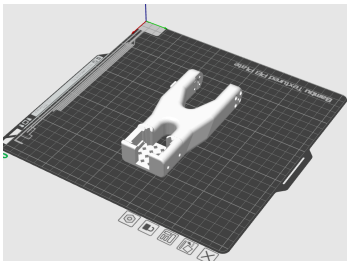
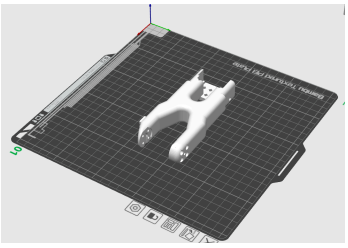
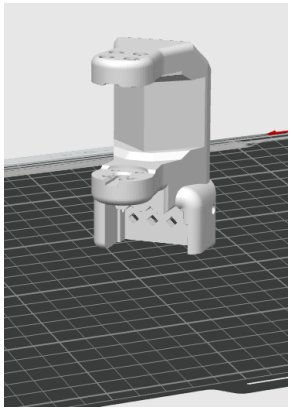
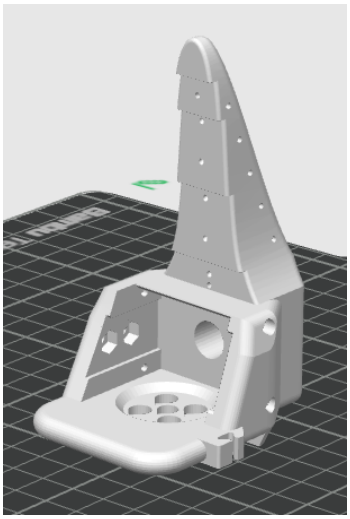
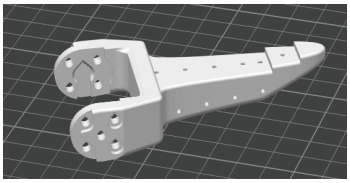
Elbow Flex	1:191	−96.83° to +96.83°	193.66°	115.8 mm	Pitch axis joint controlling the forearm extension and retraction angle.	
Wrist Flex	1:147	−95.00° to +95.00°	190.00°	135.0 mm	Controls wrist pitch (up and down tilting of the end-effector).	
Wrist Roll	1:147	−157.21° to +162.79°	320.00°	63.7 mm	Controls wrist roll (rotational orientation of the gripper assembly).	
Leader controller	1:147	−10.00° to +100.00°	110.00°	27.6 mm	Actuates the parallel mechanical jaw end-effector for grasping objects. This one has a trigger for finger to control	

Table . Follower hardware specifications

Arm Axis	Gear Ratio	Joint Limits (Degrees)	Total Range	Link Distance	Description	Visual Overview
Base / Shoulder Pan	1:345	−110.00° to +110.00°	220.00°	73.5 mm	Rotates the entire arm assembly horizontally around the Z-axis.	
Shoulder Lift	1:345	−100.00° to +100.00°	200.00°	64.7 mm	Pitch axis joint located at the shoulder. Handles the primary vertical lifting load against gravity.	
Elbow Flex	1:345	−96.83° to +96.83°	193.66°	115.8 mm	Pitch axis joint controlling the forearm extension and retraction angle.	

Wrist Flex	1:345	−95.00° to +95.00°	190.00°	135.0 mm	Controls wrist pitch (up and down tilting of the end-effector).	
Wrist Roll	1:345	−157.21 ° to +162.79°	320.00°	63.7 mm	Controls wrist roll (rotational orientation of the gripper assembly).	
Gripper	1:345	−10.00° to +100.0 0°	110.00°	27.6 mm	Actuates the parallel mechanical jaw end-effector for grasping objects.	

In terms of its physical operating capabilities, the SO-ARM101 configuration used in this project has an effective working reach of approximately 300 mm. Within this range, the practical operating workspace is approximately 100–280 mm from the robot base, where the arm can perform the intended tabletop manipulation tasks. Its estimated payload capacity is approximately 300 g at the maximum working

reach. The follower arm is equipped with a gripper that has an approximate maximum opening width of 50–54 mm, allowing it to grasp relatively small objects such as cubes, pens, small bottles, and other tabletop items. These specifications provide a practical reference for determining the size, weight, and placement of objects used in the experiments.

Although the leader and follower arms share a similar mechanical structure and can use the same servo product family, their gear-ratio configurations are different because they serve distinct purposes. The follower arm uses six servos with an identical 1:345 gear ratio. This configuration prioritizes output torque, allowing the arm to support its own structure and manipulate objects during task execution. Since the follower is primarily controlled by software rather than directly manipulated by a human operator, the additional resistance associated with a higher gear-reduction ratio is less of a concern.

In contrast, the leader arm is manually operated by a human demonstrator during data collection. Using a high gear-reduction ratio across all joints would make the arm more difficult to move manually due to the increased resistance associated with the transmission. To provide a smoother and more responsive teleoperation experience, the leader uses three different gear ratios according to the requirements of individual joints. Specifically, joints 1 and 3 use a 1:191 gear ratio, joint 2 uses a 1:345 ratio, and joints 4, 5, and 6 use a 1:147 ratio. The lower reduction ratios on selected joints improve manual back-drivability and reduce the effort required from the operator when demonstrating manipulation movements. Meanwhile, the higher reduction ratio is retained at the shoulder-lift joint to provide sufficient torque to support the arm structure. This configuration balances manual operability with the mechanical requirements of the leader arm.

Another notable difference between the two arms is their end-effector design. The follower arm is equipped with a conventional gripper consisting of movable jaws, which physically interacts with objects during manipulation. In contrast, the leader arm uses a pistol-grip-like handle with a trigger mechanism instead of a conventional object-grasping gripper. The human operator holds the handle and pulls the trigger using their finger to control the opening and closing of the follower's gripper. This design allows the operator to manipulate the leader comfortably while controlling grasping actions through a familiar hand movement. Consequently, the two end-effectors are designed around their respective functions: the leader provides an ergonomic human input interface, whereas the follower provides the physical mechanism for interacting with the environment.

The project acknowledges that the original SO-ARM design was developed through a collaboration between TheRobotStudio and Hugging Face. However, the publicly available design files primarily provide 3D STL models intended for 3D printing, without comprehensive engineering documentation specifying dimensions, joint rotation angles, working ranges, or detailed 2D technical drawings.

To address this documentation limitation and provide clearer references for the hardware configuration, the project team independently generated 2D technical drawings from the original 3D models using FreeCAD. The models were projected into multiple orthographic views, after which the drawings were edited and the relevant dimensions were measured by the team.

Therefore, **Figures [X]–[Y]** present the resulting 2D drawings of the SO-ARM101's 3D-printed structural components, including multiple views and dimensional measurements such as length and width. *These*

drawings and measurements represent the project's own technical documentation and were not provided by the original developers.

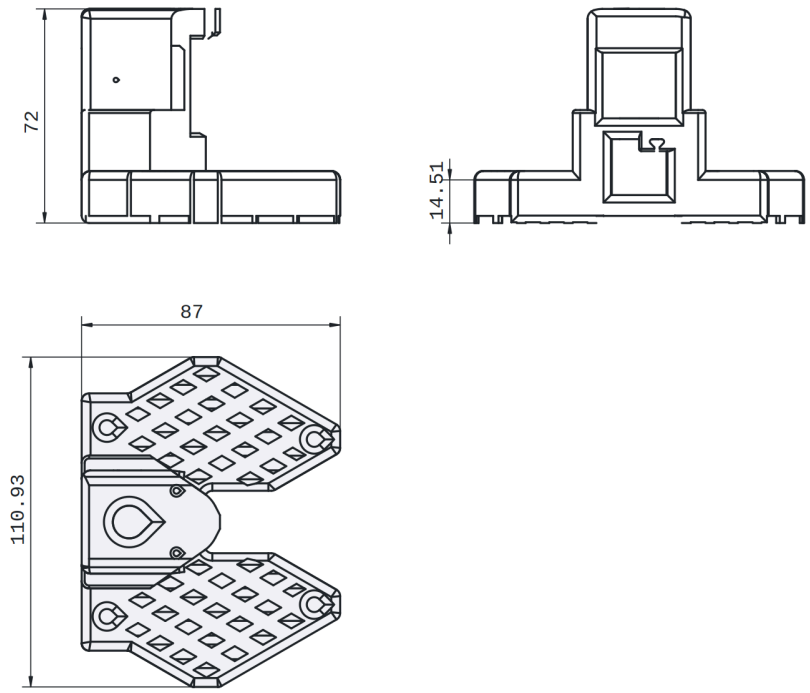


Figure X. 2D Technical Drawing of the Base Assembly.

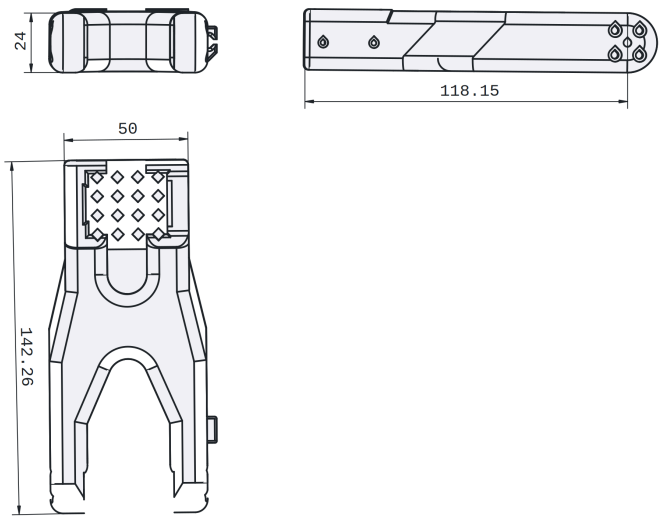


Figure X. 2D Technical Drawing of the Shoulder Lift and Elbow Flex Component.

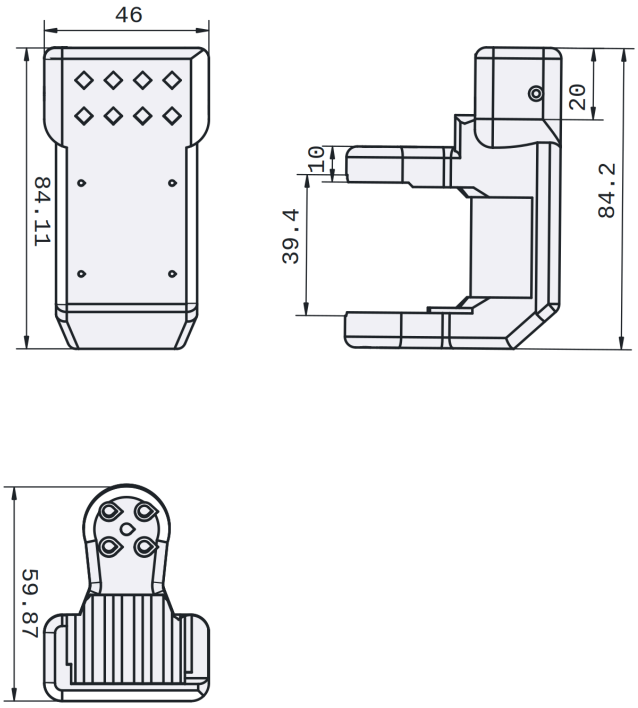


Figure X. 2D Technical Drawing of the Wrist Flex Component.

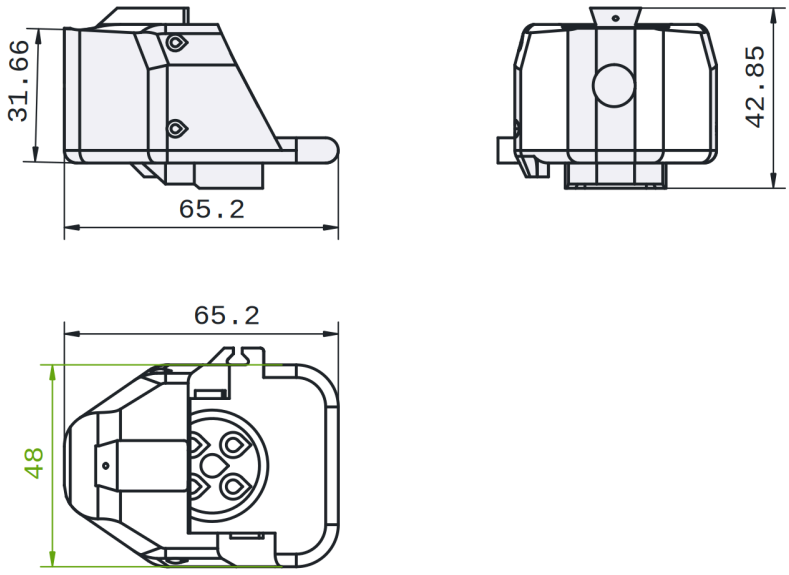


Figure X. 2D Technical Drawing of the Leader Arm's Wrist Roll Component.

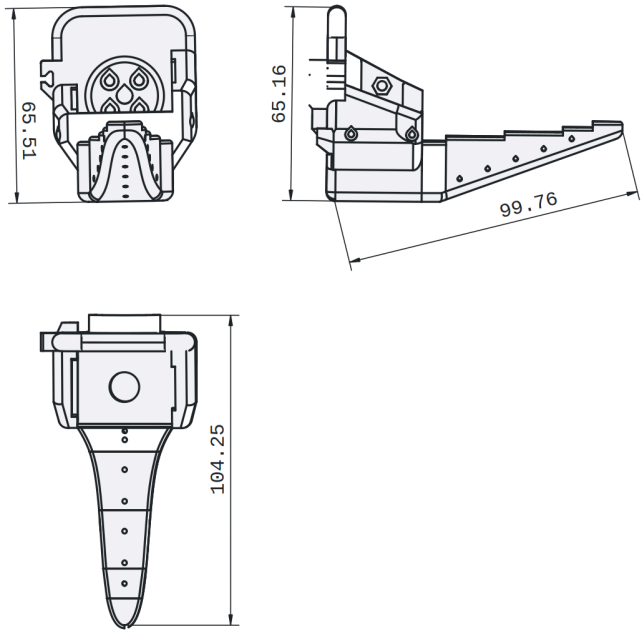


Figure X. 2D Technical Drawing of the Follower Arm's Wrist Roll Component.

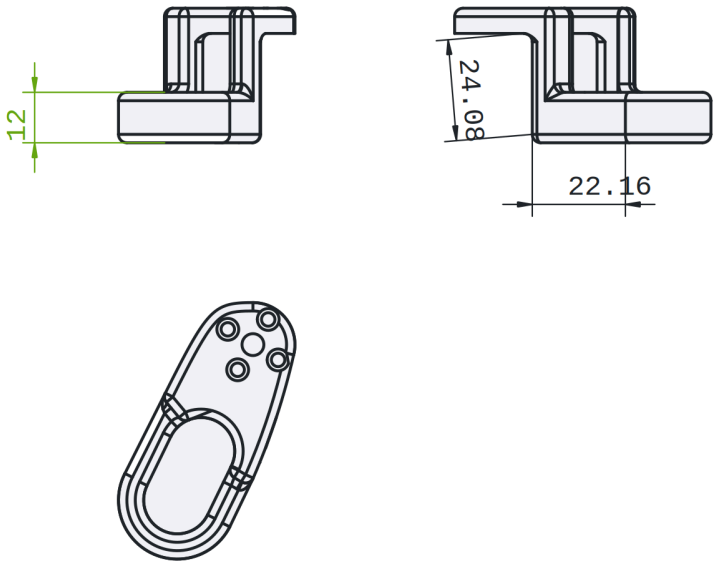


Figure X. 2D Technical Drawing of the Leader Arm’s gripper trigger .

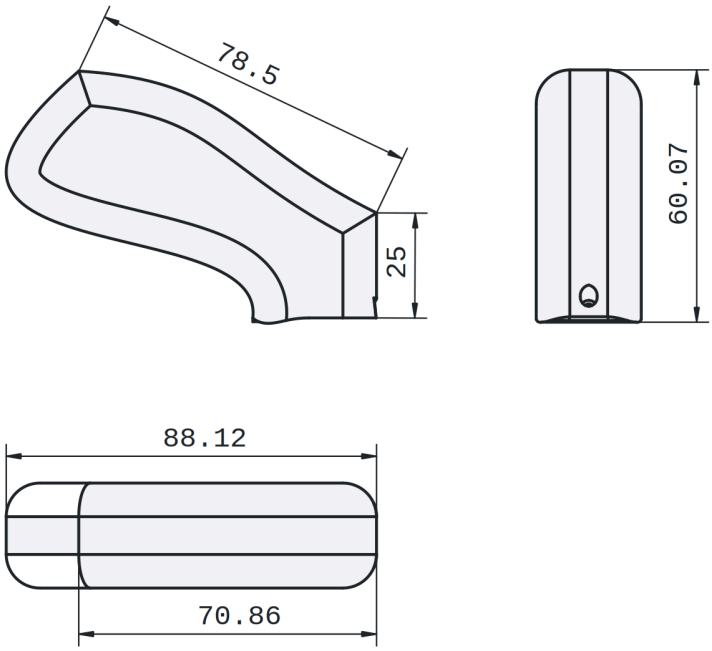


Figure X. 2D Technical Drawing of the Leader Arm's end-factor.

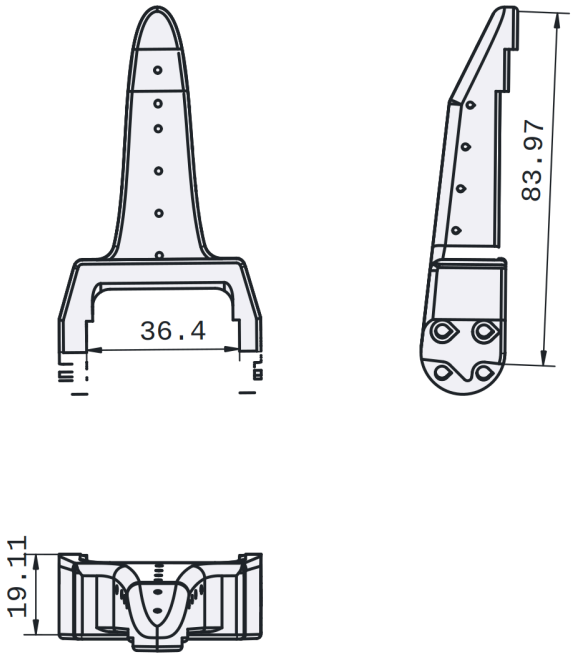


Figure X. 2D Technical Drawing of the Gripper.

Suitability of SO-ARM101 for our Project. The suitability of SO-ARM101 for this project is evaluated based on the target manipulation tasks, available resources, hardware accessibility, and compatibility with the selected robot-learning frameworks. The primary tasks investigated in this project are Desk Stationery Decluttering and Halogen Lampshade Tray Packaging. Both are tabletop manipulation tasks that primarily require a single robotic arm to identify, grasp, transport, and place objects at designated locations. Therefore, a compact single-arm platform is sufficient to support the intended experiments without requiring the additional complexity and cost of a bimanual or industrial robotic system.

One of the *main advantages* of SO-ARM101 is its affordability and open-source design. Given the project's limited financial resources and the team's relatively limited experience in mechanical and robotic hardware engineering, constructing and maintaining an expensive research-grade platform, such as the Franka Emika Panda, would introduce considerable financial and technical challenges. In contrast, SO-ARM101 provides publicly available assembly instructions, hardware designs, and community-developed tutorials, making the process of sourcing components, 3D printing, assembly, and initial configuration considerably more manageable. Furthermore, its integration with the Hugging Face LeRobot ecosystem provides native support for common robot-learning workflows, including teleoperation, demonstration collection, and policy deployment. This reduces the amount of time and effort required for hardware development, allowing the team to allocate more resources to the primary research activities, particularly VLA model integration, training, and evaluation.

Another *important advantage* is its compatibility with lightweight robot-learning policies. Algorithms and architectures such as Action Chunking with Transformers (ACT), diffusion-based policies, and SmolVLA can be trained and evaluated on the SO-ARM101 platform through the LeRobot ecosystem. This makes it a suitable experimental platform for comparing different approaches to learning robotic manipulation under relatively constrained hardware conditions. Additionally, its compact size and portability simplify transportation, workspace arrangement, and repeated experimentation. Its relatively small gripper is also appropriate for interacting with common tabletop objects, such as stationery items and small packages, which are relevant to the manipulation tasks investigated in this project.

Despite these advantages, SO-ARM101 also introduces several *limitations* that restrict the range of experiments that can be performed. As a low-cost robotic platform, its mechanical capabilities are considerably more limited than those of industrial or research-grade robotic arms. Its relatively short working reach, approximately 300 mm, and limited payload capacity restrict its ability to manipulate large, heavy, or distant objects. Consequently, tasks involving heavy lifting, large-scale object manipulation, or operations requiring an extensive working workspace are outside the intended scope of this project. Furthermore, the use of low-cost servo motors may introduce limitations in positioning accuracy, repeatability, and mechanical consistency, which should be considered when interpreting experimental results.

These *limitations* directly influence the selection and design of the project's manipulation tasks. Desk Stationery Decluttering and Halogen Lampshade Tray Packaging are selected because their object sizes, weights, and workspace requirements can be accommodated by the SO-ARM101 platform. Both tasks provide sufficient opportunities to investigate visual perception, object localization, grasping, and action execution while remaining within the mechanical capabilities of the hardware. Therefore, although SO-ARM101 does not provide the versatility or performance of more advanced robotic platforms, its

affordability, accessibility, and compatibility with the selected software ecosystem make it a practical and appropriate choice for investigating lightweight VLA policies in real-world tabletop manipulation scenarios.

Servo configuration. The SO-ARM101 robotic arms used in this project are equipped with Feetech STS3215-C001 bus servos (Fig …) , which are metal-geared digital servomotors designed for robotic applications. Each servo operates at a nominal voltage of 7.4 V (with a supported operating range of 4.8–7.4 V) and provides a stall torque of approximately 19 kg·cm at 7.4 V. The motors use a TTL communication interface, a 25-tooth spline, and an integrated digital controller, enabling position control through serial communication. Each unit weighs approximately 70.5 g and is supplied with a 30 cm cable. These servos are used to actuate the joints and gripper of the SO-ARM101, providing the torque and controllability required for the project's tabletop manipulation experiments.



Figure . Robot Servo STS3215 7.4V

Bus Driver configuration. To enable serial control of the bus servos employed in the system, the RIRIHI DC 9–12.6 V Serial Bus Servo Control Module was selected (**Fig.** …). This module provides a UART-based communication interface between the host controller and ST/SC bus servos, allowing control commands to be transmitted through a serial bus. The module operates within an input voltage range of DC 9–12.6 V and is supplied through a 5.5 × 2.1 mm DC power jack. Its compact dimensions of 42 × 33 mm, together with 2.5 mm mounting holes spaced 37 × 28 mm apart, facilitate integration into a space-constrained hardware arrangement. The module is capable of controlling up to 253 ST/SC bus servos simultaneously, subject to the capacity of the connected power supply. Within the proposed system, the bus driver serves as the interface responsible for transmitting control signals to the bus servos.

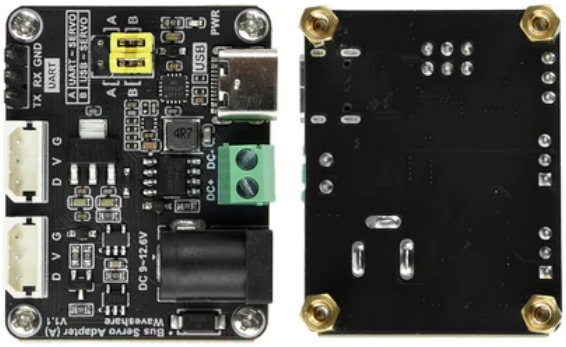


Figure. Bus Driver Servo Controller

USB HUB configuration. Owing to the limited number of directly available USB interfaces on the host system, the OTN-8121 7-in-1 USB 3.2 Gen 1 hub was incorporated into the hardware configuration (**Fig.** …). The hub provides six USB 3.0 ports and one RJ45 Ethernet port, thereby expanding both the USB connectivity and the wired networking capability of the system. The USB 3.0 ports support data transfer rates of up to 5 Gbps, whereas the RJ45 interface provides a 100 Mbps Ethernet connection. An auxiliary DC 5 V power input is required to maintain stable operation when multiple peripheral devices are connected. In addition, the hub incorporates a locking mechanism and a status display, and its construction combines a metal enclosure with a TPE cable. In the proposed configuration, the USB hub is used to connect the USB-based peripheral devices required by the system.



Figure. USB Hub 7-in-1

Camera configuration. The visual sensing subsystem consists of two camera types, each selected according to its resolution, field of view, and intended image-capture function. The first device is the ICON Q-20 webcam (**Fig. …**), which supports HD, 2K, and 4K resolutions at frame rates of up to 60 fps. The camera measures 50 × 100 × 100 mm and integrates a noise-reducing microphone, automatic focusing, 360-degree rotation, and plug-and-play compatibility with PC and laptop platforms. The second device is a 720P USB camera module (**Fig**. …), which provides a resolution of 1280 × 720 pixels at 30 fps and employs an OV9732 image sensor. This module offers a 100-degree distortion-free field of view, an adjustable-focus lens, and driver-free compatibility with Windows, Linux, Android, and macOS operating systems. In the proposed system, these cameras are employed to acquire visual data for the image-capture and recognition functions.



Figure. Webcam ICON Q-20



Figure. USB Camera Module 720P

Adapter Configuration. To provide a configurable DC power supply for the electronic components of the system, a 60 W adjustable power adapter was incorporated into the hardware configuration (Fig. …). The adapter allows the output voltage to be manually adjusted within a range of 3–12 V DC and delivers a maximum output current of 5 A, corresponding to a rated power of 60 W. An integrated LED display provides a real-time indication of the actual output voltage, enabling the supply level to be verified during operation. The adapter also incorporates several protection mechanisms, including over-voltage, over-current, short-circuit, and over-temperature protection. Its output is provided through 5.5 × 2.5 mm and 5.5 × 2.1 mm DC jacks, which are compatible with commonly used control modules.



Figure. Adapter use in the system

Hardware-to-Computer system. The hardware-to-computer system integration describes how the power supply, sensing devices, servo control modules, and robotic arms are connected to the host computer. In this configuration, the host computer functions as the central processing unit and communicates with the external hardware through the OTN-8121 7-in-1 USB hub. The USB hub is connected to the host computer through a USB 3.0 interface, which provides a data transfer rate of up to 5 Gbps and serves as the main communication path between the computer and the connected peripherals.

The power supply network begins from a 220 V AC mains source, which is connected to the 60 W adjustable DC adapter. The adapter converts the AC input into a regulated DC output within the range of 6–7.4 V, which is supplied directly to the two servo bus drivers used for the Leader and Follower robotic arms. This voltage range is compatible with the operating requirements of the servo bus control modules and ensures that both arms receive a stable power supply during operation.

The USB hub manages five data connections from the peripheral devices. Two ICON Q-20 webcams are connected to the hub to capture visual observation streams, with support for resolutions of up to 4K at 60 fps. One OV9732 USB camera module is also connected to the hub to acquire close-range visual data for computer vision and recognition tasks, providing a resolution of 720p at 30 fps and a 100-degree distortion-free field of view. In addition, the Leader and Follower servo bus drivers are connected to the hub to enable bidirectional communication between the robotic arms and the host computer.

The Leader SO-ARM101 is physically manipulated by the human operator. Its servo bus driver reads the joint angles and position feedback from the servos and transmits this information to the host computer. The host computer then processes the received motion data and sends corresponding control

commands to the Follower servo bus driver. The Follower SO-ARM101 receives these commands through its servo bus driver and reproduces the motion of the Leader arm, enabling teleoperation and imitation-based control.

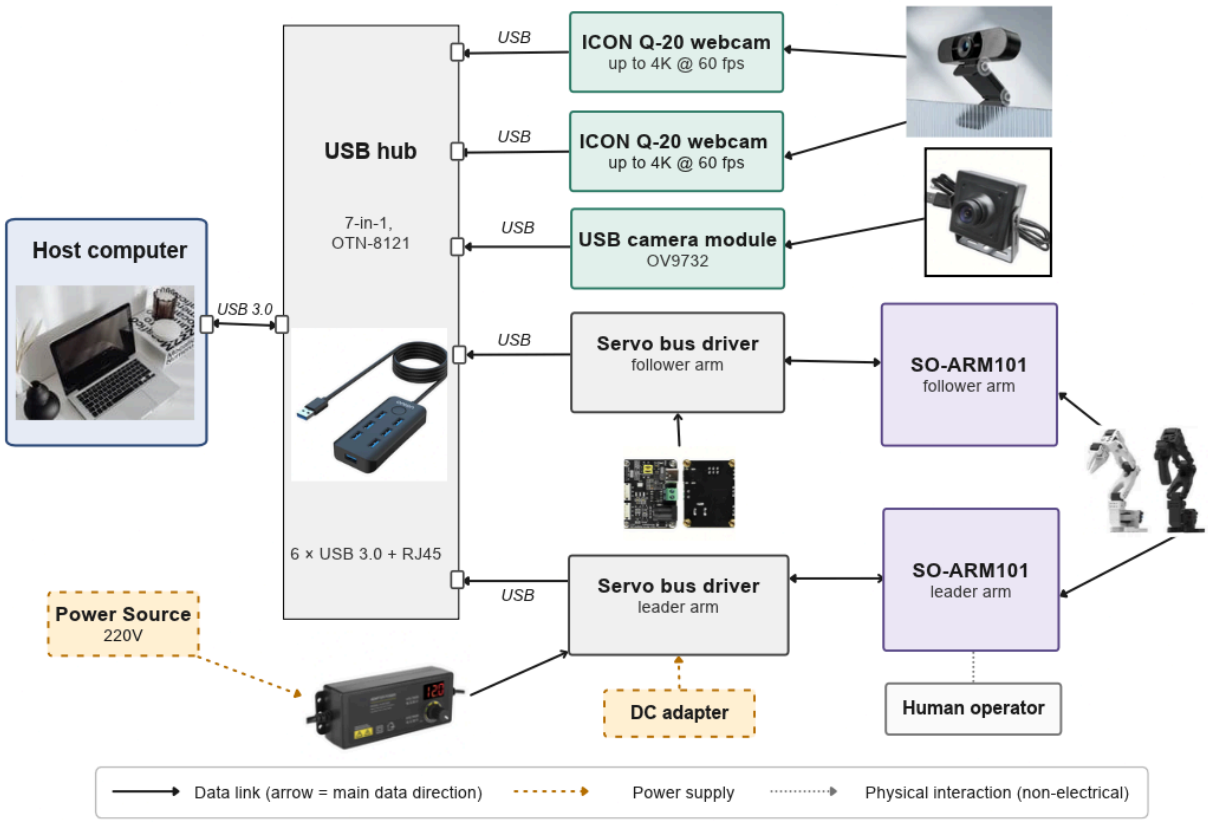


Figure. Computer-to-robot-arm system connection diagram

b) Desk Stationery Decluttering Setup

The Desk Stationery Decluttering task is a tabletop manipulation scenario in which a robotic arm collects scattered stationery items, such as pens, pencils, and erasers, and places them into a designated storage container, such as a pencil case. In this project, the task is defined more specifically to provide a consistent and reproducible environment for training and evaluating robotic policies.

The experimental setup consists of four physical objects: three stationery items and one rectangular pencil case. The stationery items include a brush pen, a ballpoint pen, and an eraser, while the pencil case serves as the target container. At the beginning of each trial, the three stationery items are placed at different positions within the SO-ARM101's working area. The objective is for the robotic arm to identify, grasp, and transfer all three items into the pencil case, which is positioned at a designated location within the same workspace. The initial positions and orientations of the stationery items can be varied between trials to introduce spatial variation and reduce reliance on memorized object positions.

The dimensions and physical characteristics of the objects used in this experiment are presented in **Figure X.**

Motivation and Real-World Relevance. The selection of this task is motivated by the everyday need to organize personal workspaces, particularly study desks. Stationery items are frequently used by students and office workers, but they are also easily left scattered across a desk after studying or completing daily tasks. Although collecting these items is a relatively simple activity for humans, it is repetitive and may be postponed when users have limited time or motivation. Survey findings provide some supporting evidence for this broader problem. According to the American Cleaning Institute's 2022 College Cleaning Survey, conducted among 500 US college undergraduates, 53% of respondents identified a lack of motivation and 50% identified a lack of time as factors preventing them from keeping their dormitories or rooms clean [cite]. Furthermore, its 2024 survey of 500 US college seniors and rising seniors found that 45% considered having a clean living space beneficial for concentrating on schoolwork, while 33% identified decluttering their homes as a habit they wished they had developed before college [cite2]. Although these surveys address general living-space cleanliness rather than desk organization specifically, they indicate that maintaining an organized personal environment is a relevant concern among college students.

From an application perspective, a compact robotic arm integrated into a desk environment could provide assistance with repetitive object-organizing activities. For example, a student could initiate a decluttering task after finishing a study session, allowing the robotic arm to collect selected stationery items and return them to a designated container. This concept represents a potential application of affordable desktop robotic assistance, particularly for small and repetitive manipulation tasks. However, the proposed experiment focuses on demonstrating the feasibility of this specific manipulation workflow rather than developing a complete autonomous desk-cleaning product.

The task also provides a meaningful evaluation scenario for Vision-Language-Action policies. The robot must visually distinguish objects with different shapes and dimensions, determine suitable grasping positions, coordinate its movements, and place the objects into a relatively narrow container opening. These requirements introduce several manipulation challenges within a controlled tabletop environment. In particular, the difference between the thin ballpoint pen and the wider brush pen allows the experiment to examine whether a learned policy can handle objects with different grasping characteristics. By varying their initial positions and orientations, the task can additionally evaluate the policy's ability to respond to changes in the visual arrangement of the workspace.

Existing Solutions and Positioning of the Proposed Setup. Robotic platforms and research projects show that tabletop object manipulation can be automated. The Rotrics DexArm is a commercial desktop arm for drawing, laser engraving, and 3D printing, with optional gripper, suction, and vision modules, but it targets multifunctional fabrication rather than open-source VLA research [cite]. Tabletop HandyBot is an open-source tabletop assistant that performs pick-and-place from user instructions using an AR4 arm, RealSense D435 camera, and vision-language components, with a BOM of about USD 2,300—demonstrating language-guided manipulation but at greater complexity and cost than the compact SO-ARM101 used here [cite]. Most directly related, CleanBot (2026 James Dyson Award) uses an SO-101 arm for table-clearing via human demonstrations and imitation learning, showing the

potential of low-cost arms for everyday organizing [ciet]. This supports the practical relevance of the proposed task, though it also indicates that SO-101-based table clearing has already been explored.

In comparison, the proposed setup adopts SO-ARM101 because of its affordability, compact form factor, and integration with the Hugging Face LeRobot ecosystem. Rather than developing a complete commercial desk-cleaning product, this project uses stationery decluttering as a controlled real-world scenario for investigating and comparing lightweight VLA policies. The setup allows the evaluation of visual understanding, grasping, and object-placement capabilities using inexpensive and commonly available objects. It also provides a reproducible environment in which different VLM backbones can be integrated into SmolVLA and evaluated under the same physical task conditions.

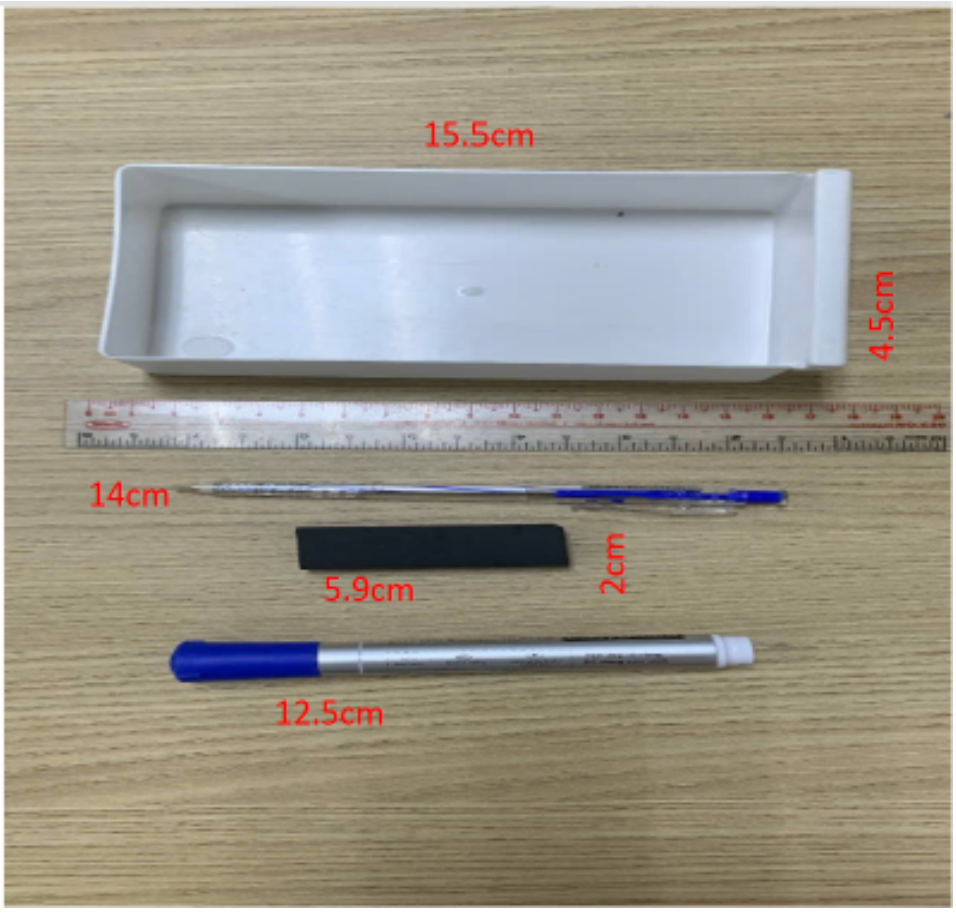


Fig. Objects for Desk Stationary Decluttering Task

Objects and Physical Specifications. The experimental setup contains three stationery items and one storage container. Each object is selected to introduce different physical characteristics and grasping requirements while remaining within the operating capabilities of the SO-ARM101 platform.

The first object is a rectangular white plastic pencil case, which serves as the target container for the collected stationery. It measures approximately 15.5 cm in length, 4.5 cm in width, and 5 cm in height. Its open-top design allows the robotic arm to place objects inside by approaching from above. Plastic is

selected because it is lightweight, relatively durable under normal experimental handling, and inexpensive to replace if damaged during repeated trials.

The second object is a conventional plastic ballpoint pen measuring approximately 14 cm in length and 1 cm in width. Its relatively thin cylindrical body presents a more challenging grasping condition because the gripper must make sufficient contact with a narrow surface without pushing the pen away or causing it to roll. This object is therefore useful for evaluating the policy's ability to handle relatively small and thin objects.

The third object is a brush pen measuring approximately 12.5 cm in length and 2.5 cm in width. Compared with the ballpoint pen, its wider body provides a larger contact surface for the gripper, making it relatively easier to grasp. Including both pen types allows the experiment to examine how differences in object dimensions influence manipulation performance.

The final object is a black rectangular eraser measuring approximately 6 cm in length and 2 cm in width. Unlike the elongated pen-shaped objects, its shorter and more compact structure introduces a different grasping geometry. Its inclusion provides additional variation in object shape and orientation, allowing the policy to demonstrate its ability to manipulate objects with different physical characteristics.

All stationery items remain below 100 g in weight, making them suitable for the payload limitations of the SO-ARM101. The full image of set is illustrated by Fig …

Demonstration Scenario. The demonstration scenario is designed around a typical student study session. After completing their work, a student leaves several stationery items scattered across the desk and wishes to organize the workspace without manually collecting each item. The student accesses the robot's control interface, selects the Desk Stationery Decluttering task, and submits the corresponding command. The trained policy then processes the visual observations of the tabletop and controls the SO-ARM101 follower arm to collect the scattered stationery items and place them into the pencil case. Upon successfully completing the task, the stationery is stored in the designated container, leaving the working surface more organized.

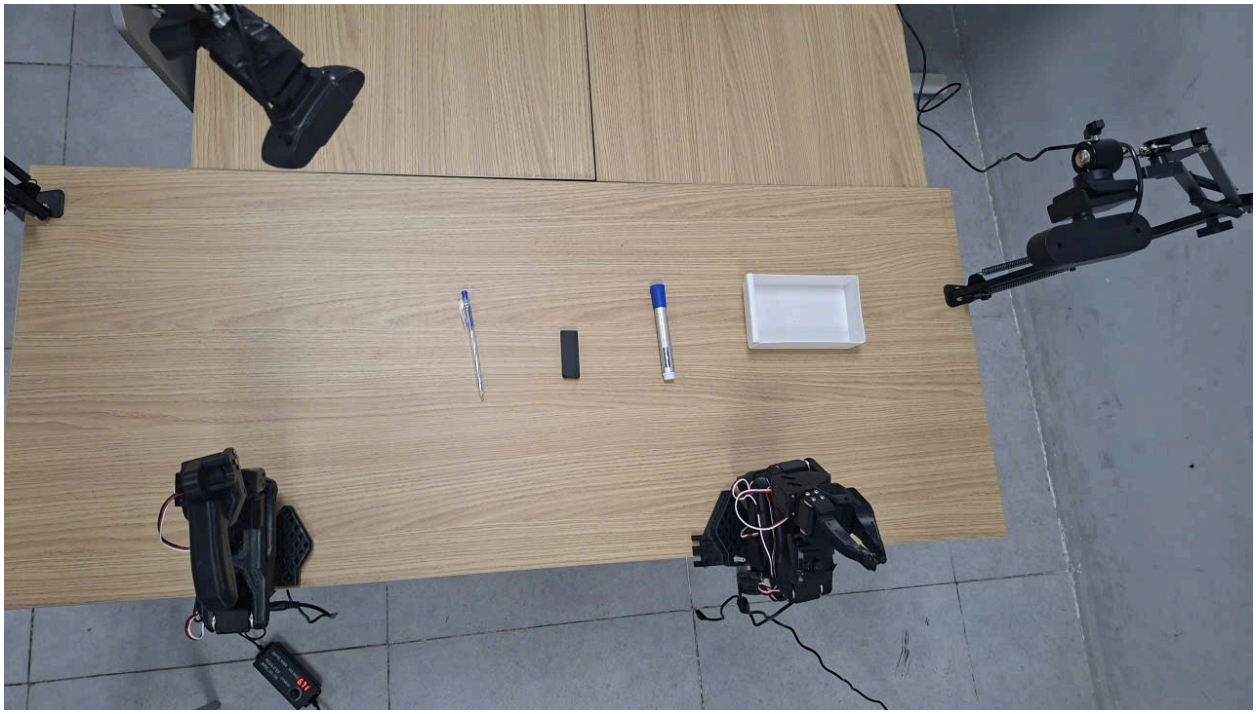


Fig. Environment and hardware setup for Desk Stationary Decluttering task

c) **Halogen Lampshade Tray Packaging (Inspecting)**

3. Data Collection and Preprocessing

[Describe how you will collect data, including the sources of data, methods of collection, and any selection criteria. Information about the sample size, diversity, and representativeness of the data should be included.]

[Detail how the data will be processed before analysis, including cleaning data, handling missing data, transforming data, and normalization.]

a) Data Collection

Source. Our project goal is to test weather doing the backbone swapping for SmolVLA actually improve performance or not with evaluation on two task Desk stationary decluttering and … . To do so, we have to collect 2 separate pieces of data, for simplicity of the name, from now on, we will call it Pick-and-Place and Long Horizon task.

As we don't intend to set up stimulation environment like Issac Sim or Mujoco to avoid Sim-to-Real problem, all data for training and evaluation have to be real-world data, which is much more realistic than stimulation. However, open real world dataset like Open X, Droids, … are not natively on So-arm101, which is ou main hardware, the different in hardware and calibration make it extremely

dfficulut to train and performan and evaluate a policy well. Moreover, as we use 3 camera with specific objects are stationary learning objects and halogen lightbub, these are not present commnuniy in hugging face. Therefore, we decided to collect the data by ourselves.

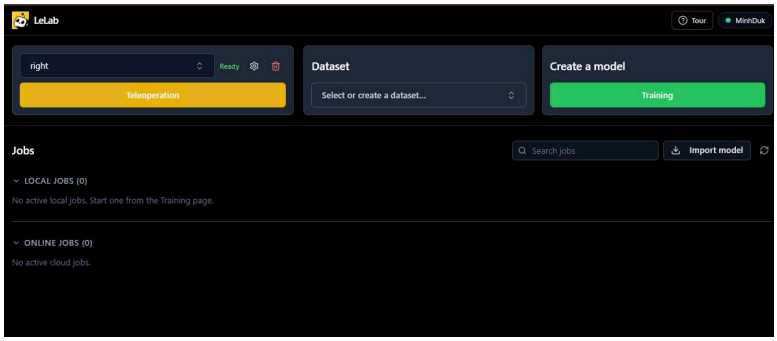


Fig . Lelab Web UI

Thanks to the development of community support robotic like lerobot and lelab, we can easily collect data by using these. Lelab, shown in **Fig**, is a the platform we mainly use to collect these 2 real-world dataset. LeLab is a web app that puts the full LeRobot workflow — calibrate, teleoperate, record, train, replay — into a single browser UI and dedicated to many types robotic arm of lerobot. A web-native front door to LeRobot, designed so newcomers can get from "unboxing" to "training their first policy" in minutes.

Collection methods. As we say above, the main platform we is Lelab. However, choosing the right platform is only the first step, the next one is connect our so-arm101 to the platform and do calibration. First we do the find port to know the connection port to the so-arm, next we strat calibrate by spinning the joist - server in it for it to record the range of activation. Once this is complete, we add 3 views of cameras, which are cam_head - on the top, cam_wrist on the arm and cam_side is camera looking at the side to the files. Once this is done, the calibration files is save and can be use for that arm later. **Fig ...** illustrate the screen for calibration arm.

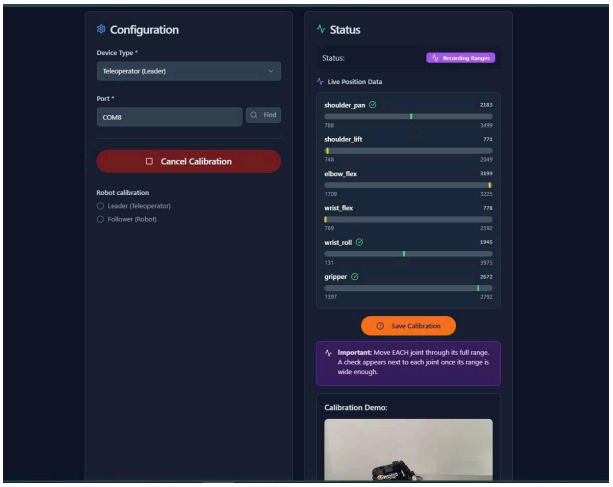


Fig. Calibration step before collecting data

Once the hardware and environmental setup has been completed, the data collection phase is configured through the LeLab interface, as illustrated in Figure [X]. Before recording begins, several parameters must be specified, including the dataset name, task description, number of demonstration episodes, episode duration, and reset duration. The task description serves as the natural-language instruction associated with the demonstrations and is subsequently used as the task prompt for the Vision-Language-Action (VLA) policy.

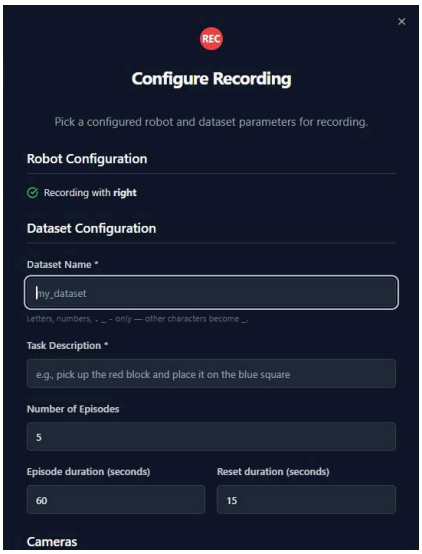


Fig . Lelab configuration for data collection

Since the primary learning approach employed in this project is a form of Behavioural Cloning (BC), the policy must first be provided with demonstrations of the desired robot behaviour. These demonstrations are collected through a leader–follower teleoperation mechanism. During operation, a human demonstrator manually controls the leader arm by holding its pistol-grip-shaped end-effector, with a finger positioned on the trigger to operate the gripper. Once an episode begins recording, the demonstrator moves the leader arm to approach, grasp, transport, and release the target objects according to the task requirements.

Simultaneously, the follower arm reproduces the corresponding movements of the leader arm, thereby executing the demonstrated manipulation sequence in the physical environment. The resulting observations and robot actions are recorded as demonstration data for policy training. However, the follower's movements are not expected to match the leader's movements perfectly. Small discrepancies may occur due to differences in calibration and mechanical alignment between the two arms.

One additional limitation of this teleoperation arrangement concerns the feedback received by the human demonstrator. Since the leader and follower operate as separate mechanisms, the demonstrator does not directly perceive the physical resistance encountered by the follower when interacting with objects. Although this limitation does not prevent the collection of demonstrations for the current tasks, it may affect manipulation consistency and place additional mechanical demands on the follower's actuators during repeated interactions. Therefore, it should be considered when evaluating the reliability and long-term operation of the system.

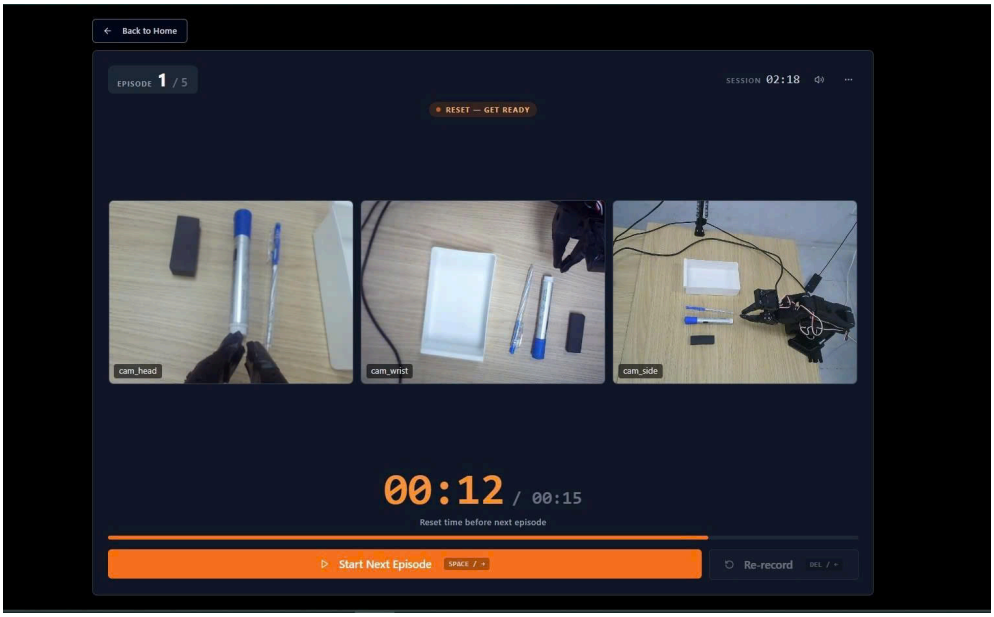


Fig . Lelab UI for data collection

To ensure consistent data collection, two team members are assigned to each recording session. One member operates the LeLab interface, managing the recording process by starting and stopping individual episodes, while the other performs the teleoperation demonstrations using the leader arm. After approximately 50 demonstrations, the two members exchange responsibilities to distribute the workload and reduce the influence of individual operating styles on the collected data. This process is repeated until the target of 300 episodes is reached for each task.

In terms of project scheduling, approximately one working day is allocated to collecting the demonstrations for each dataset, followed by another day for data processing and cleaning. The additional processing stage is necessary to inspect the collected recordings and prepare the datasets for subsequent policy training.

Demonstration Procedure for Desk Stationery Decluttering (Pick-and-place). For this task, which follows a pick-and-place procedure, the dataset is configured with the name Cleaning. The task description is set to *"Pick the pens and rubber, put them into the whitebox."* A total of 300 demonstration episodes are collected, with an episode duration of 120 seconds and a reset duration of 15,000.



Fig . Demonstration for collecting data for Pick-and-Place task

For the Desk Stationery Decluttering task, a human demonstrator operates the leader arm to perform the pick-and-place sequence for each stationery item. Fig . shown the demonstration consists of five main stages:

1. **Target Approach:** The demonstrator first raises the leader arm and positions it in an inverted U-shaped configuration. The arm is subsequently lowered to bring the gripper into proximity with the selected stationery item on the tabletop.
2. **Object Grasping:** The demonstrator operates the trigger integrated into the leader end-effector using the left index finger, thereby controlling the opening and closing of the follower gripper. The gripper is opened to accommodate the target object and then closed to establish a grasp.
3. **Object Lifting and Transportation:** Once the object has been grasped, the demonstrator raises the arm to lift it from the tabletop. The base is then rotated to orient the arm towards the white pencil case, transporting the object to the designated placement area.

4. **Placement Positioning:** The demonstrator adjusts the arm position and lowers the gripper towards the open central area of the pencil case. The gripper is carefully aligned above the container to ensure that the object can be released within its boundaries.
5. **Object Release:** The demonstrator operates the trigger to open the follower gripper, releasing the object into the pencil case and completing one pick-and-place cycle. The same sequence is subsequently repeated for the remaining stationery items until all three objects have been transferred into the container.

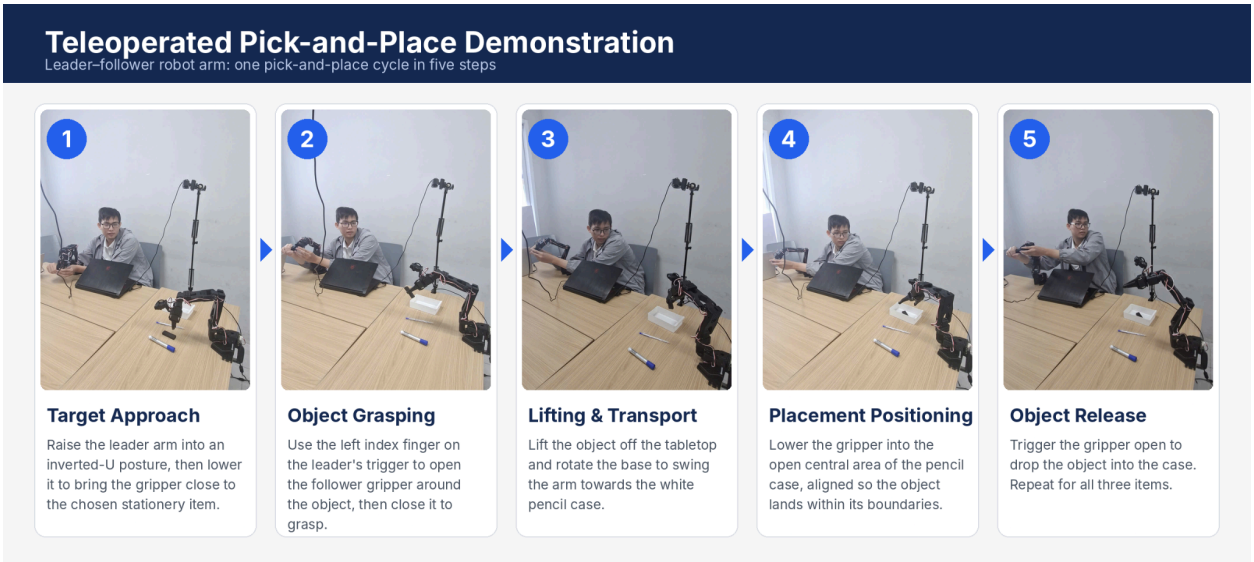


Fig . Data collection cycle

Demonstration Procedure for Halogen Lampshade Tray Packaging (Inspecting).

Selection Criteria

The basic criteria for saving the dataset is that it must not have broken files or missing camera views. As if we include these in the training data, the train might be broke and lead to the uncomplete training. Moreover, for any demonstration that the controller fail to grapes the objects too many time (more than 2), then, will it also be exclude from the final dataset as we dont want the policy to leanr those fail behaviour. All the data will be process and filter before finally feed into VLA for learning.

Sample Information.

4. Feature Selection and Engineering

[Describe the process of selecting and creating features that will be used for the machine learning model, as well as the reasons for these choices.]

SmolVLA.

5. Method

6. Model Training and Validation

[Describe the machine learning models considered and the reasons for the final model choice, including technical specifications and configurations of the model.]

[Detail how the model will be trained and validated, including information on data splitting methods, cross-validation techniques, and the metrics used for evaluation.]

SmolVLA.

7. Evaluation Metrics

[Identify the metrics that you will use to assess the performance of the model, such as accuracy, F1 score, ROC AUC, etc.]

SmolVLA.

8. Implementation Plan

[Describe in detail how the model will be implemented, as well as any tools or technologies used to support this process.]

SmolVLA.

9. Ethical and Safety Considerations

[Discuss ethical issues that may arise when applying machine learning, including data privacy, accuracy, and how to handle data fairly.]

SmolVLA.

IV. Methodology (sample 2)

1. Project Objectives and RL Challenges

[Clearly identify the project objectives and the specific challenges that reinforcement learning aims to address, such as learning to optimize decisions in an uncertain environment.]

SmolVLA.

2. Environment Setup

[Describe the environment in which the reinforcement learning agent will operate, including parameters, states, and rewards.]

3. Agent Design and States

[Describe the design of the RL agent, including states, actions, and policies it may adopt.]

4. Reward System

[Detail the reward system, including how rewards and penalties are defined to guide the behavior of the agent.]

5. RL Algorithm

[Describe the reinforcement learning algorithm used, including any variations from standard methods such as Q-learning, Deep Q-Networks (DQN), Policy Gradients, etc. Mention how the algorithm will be applied to allow the agent to learn from experience.]

6. Training and Evaluation Strategy

[Describe the training and evaluation process for the agent, including how and how often the agent will explore and exploit the environment to learn. Information on how the performance and learning of the agent are tested in different situations should be included.]

7. Implementation Details

[Provide details on the implementation, including the software development environment, tools, and libraries used.]

8. Ethical and Safety Considerations

[Discuss ethical and safety issues related to the deployment and testing of the RL agent, especially in scenarios that might impact the real world.]

[Note: By adjusting and supplementing the content appropriately for the "Methodology" section, you will help ensure that this part accurately reflects the approach and specific challenges of the AI capstone project.]

V. System Design and Implementation

[Provide the final system design and implementation following the template as part II in the Report #5]

1. AI Model Integration

[Describe how the AI model is integrated into the larger system, including the interface and data exchange between the AI model and other system components.]

SmolVLA.

2. Data Flow and Processing

[Illustrate and describe the data flow through the system, from data collection, processing, to data analysis and result presentation.]

SmolVLA.

3. Deployment Strategy

[Detail how the system will be deployed, including the deployment environment and deployment procedures.]

SmolVLA.

4. Scalability and Maintenance

[Discuss how the system is designed to handle growth and future maintenance, including system updates, scalability, and version management.]

SmolVLA.

[Note: By providing a detailed view of the AI solution's design and implementation in this section, you will help readers understand the technical and systematic approach you have applied to your capstone project.]

VI. Results and Discussion

[Provide the final results and discussion following the template as part II in the Report #6]

1. Results and Analysis

[Present the results obtained from executing the project, including numerical data, graphs, diagrams, and supportive imagery. This may include the outcomes of machine learning models, such as accuracy, confusion matrices, ROC curves, and other evaluation metrics.]

[Provide a detailed analysis of the results obtained, explaining their significance and impact on the project. This might include comparing the performance of different models, error analysis, and interpreting the features of the model.]

SmolVLA.

2. Discussion

[Discuss the results, including how they address the research questions and hypotheses, their alignment with results from related studies, and the broader context of the project. Mention any limitations or challenges encountered during the project.]

SmolVLA.

3. Recommendations

[Offer recommendations based on the findings, including suggestions for further research, system improvements, or practical applications.]

SmolVLA.

4. Personal Reflection (Optional)

[A section reflecting personally on the project process, lessons learned, and how improvements could be made in the future.]

SmolVLA.

[Note: The "Results and Discussion" section is an essential part of the capstone project document, where you evaluate the success of your project based on data and analysis. This is also an opportunity for you to reflect and discuss the larger significance of the work you have done, as well as to identify any opportunities for further research or development.]

VII. Conclusion

[Provide the final conclusion following the template as part II in the Report #7]

1. Summary of Findings

[Summarize the main findings of the project, including how the research questions have been answered and how the project goals were achieved.]

SmolVLA.

2. Contributions and Reflections on the Project

[Highlight the specific contributions of the project to the field of study, community, or industry. This could include technical advancements, new knowledge, or practical applications.]

[Reflect on the project process, including what worked well and what could be improved. This is also an opportunity to acknowledge the level of challenge and learning from the project.]

[Provide some final thoughts or conclusion, expressing hopes or vision for the future of the research or the problem addressed by the project.]

SmolVLA.

3. Limitations and Future Work

[Acknowledge any limitations of the project and how they may affect the results and conclusions. This includes factors such as data, methods, or assumptions.]

[Suggest directions for future research or development based on the results and limitations of the project. This helps guide others who may build upon your work.]

[Note: The "Conclusion and Future Work" section is not only a place to summarize the completed work but also an opportunity to highlight the importance of the project and propose next steps. This helps readers understand the impact and potential of the work you have done.]

SmolVLA.

VIII. References

[1] J. Aldaco et al., “ALOHA 2: An enhanced low-cost hardware for bimanual teleoperation,” arXiv:2405.02292, 2024, doi: 10.48550/arXiv.2405.02292.

[2] A. Amini et al., “LFM2 technical report,” arXiv:2511.23404, 2025, doi: 10.48550/arXiv.2511.23404.

[3] K. E. Bekris, J. Doerr, P. Meng, and S. Tangirala, “The state of robot motion generation,” presented at the Int. Symp. Robot. Res. (ISRR), Long Beach, CA, USA, Dec. 8–12, 2024. [Online]. Available: https://arxiv.org/abs/2410.12172

[4] K. Black et al., “π0: A vision-language-action flow model for general robot control,” in Proc. Robot.: Sci. Syst. XXI, Los Angeles, CA, USA, Jun. 21–25, 2025, doi: 10.15607/RSS.2025.XXI.010.

[5] A. Brohan et al., “RT-1: Robotics transformer for real-world control at scale,” in Proc. Robot.: Sci. Syst. XIX, Daegu, Republic of Korea, Jul. 10–14, 2023. [Online]. Available: https://arxiv.org/abs/2212.06817

[6] R. Cadene et al., “LeRobot: An open-source library for end-to-end robot learning,” in Proc. 14th Int. Conf. Learn. Represent. (ICLR), Rio de Janeiro, Brazil, Apr. 23–27, 2026. [Online]. Available: https://arxiv.org/abs/2602.22818

[7] X. Chen et al., “PaLI-X: On scaling up a multilingual vision and language model,” arXiv:2305.18565, 2023, doi: 10.48550/arXiv.2305.18565.

[8] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song, “Diffusion policy: Visuomotor policy learning via action diffusion,” in Proc. Robot.: Sci. Syst. XIX, Daegu, Republic of Korea, Jul. 10–14, 2023, doi: 10.15607/RSS.2023.XIX.026.

[9] M. Deitke et al., “Molmo and PixMo: Open weights and open data for state-of-the-art vision-language models,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), Nashville, TN, USA, Jun. 11–15, 2025, pp. 91–104. [Online]. Available: https://arxiv.org/abs/2409.17146

[10] D. Driess et al., “PaLM-E: An embodied multimodal language model,” in Proc. 40th Int. Conf. Mach. Learn. (ICML), Honolulu, HI, USA, Jul. 23–29, 2023, Proc. Mach. Learn. Res., vol. 202, pp. 8469–8488. [Online]. Available: https://proceedings.mlr.press/v202/driess23a.html

[11] E. Jang et al., “BC-Z: Zero-shot task generalization with robotic imitation learning,” in Proc. 5th Conf. Robot Learn. (CoRL), London, U.K., Nov. 8–11, 2021, Proc. Mach. Learn. Res., vol. 164, 2022, pp. 991–1002. [Online]. Available: https://proceedings.mlr.press/v164/jang22a.html

[12] A. Khazatsky et al., “DROID: A large-scale in-the-wild robot manipulation dataset,” in Proc. Robot.: Sci. Syst. XX, Delft, The Netherlands, Jul. 15–19, 2024. [Online]. Available: https://arxiv.org/abs/2403.12945

[13] M. J. Kim et al., “OpenVLA: An open-source vision-language-action model,” in Proc. 8th Conf. Robot Learn. (CoRL), Munich, Germany, Nov. 6–9, 2024, Proc. Mach. Learn. Res., vol. 270, 2025, pp. 2679–2713. [Online]. Available: https://proceedings.mlr.press/v270/kim25c.html

[14] R. Knight et al., “Standard Open SO-100 & SO-101 Arms,” TheRobotStudio, GitHub repository, 2024. Accessed: Oct. 31, 2026. [Online]. Available: https://github.com/TheRobotStudio/SO-ARM100

[15] J. Kober, J. A. Bagnell, and J. Peters, “Reinforcement learning in robotics: A survey,” Int. J. Robot. Res., vol. 32, no. 11, pp. 1238–1274, 2013, doi: 10.1177/0278364913495721.

[16] Z. Li, X. Wu, H. Du, F. Liu, H. Nghiem, and G. Shi, “A survey of state of the art large vision language models: Alignment, benchmark, evaluations and challenges,” arXiv:2501.02189v7, Aug. 2026, doi: 10.48550/arXiv.2501.02189.

[17] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow matching for generative modeling,” in Proc. 11th Int. Conf. Learn. Represent. (ICLR), Kigali, Rwanda, May 1–5, 2023. [Online]. Available: https://arxiv.org/abs/2210.02747

[18] Liquid AI, “LFM2.5-VL-450M,” Hugging Face, model card. Accessed: Oct. 31, 2026. [Online]. Available: https://huggingface.co/LiquidAI/LFM2.5-VL-450M

[19] A. Marafioti et al., “SmolVLM: Redefining small and efficient multimodal models,” arXiv:2504.05299, 2025, doi: 10.48550/arXiv.2504.05299.

[20] Open X-Embodiment Collaboration, “Open X-Embodiment: Robotic learning datasets and RT-X models,” in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), Yokohama, Japan, May 13–17, 2024, pp. 6892–6903. [Online]. Available: https://arxiv.org/abs/2310.08864

[21] OpenGVLab, “InternVL2-2B,” Hugging Face, model card. Accessed: Oct. 31, 2026. [Online]. Available: https://huggingface.co/OpenGVLab/InternVL2-2B

[22] D. A. Pomerleau, “ALVINN: An autonomous land vehicle in a neural network,” in Advances in Neural Information Processing Systems 1, D. S. Touretzky, Ed. San Mateo, CA, USA: Morgan Kaufmann, 1989, pp. 305–313, presented at NIPS 1988, Denver, CO, USA, 1988. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/1988/file/812b4ba287f5ee0bc9d43bbf5bbe87fb-Paper.pdf

[23] Qwen Team, “Qwen3.5-0.8B,” Hugging Face, model card. Accessed: Oct. 31, 2026. [Online]. Available: https://huggingface.co/Qwen/Qwen3.5-0.8B

[24] H. Sawada and S. Kasahara, “PredVLA: A sub-million-parameter predictive-coding policy for robot manipulation,” arXiv:2608.26673v1, Aug. 2026, doi: 10.48550/arXiv.2608.26673.

[25] K. Sendai, T. Matsushima, and Y. Iwasawa, “MINERVA: How small can a manipulation policy be and still solve LIBERO?,” arXiv:2609.03715, Sep. 2026, doi: 10.48550/arXiv.2609.03715.

[26] M. Shukor et al., “SmolVLA: A vision-language-action model for affordable and efficient robotics,” arXiv:2506.01844, 2025, doi: 10.48550/arXiv.2506.01844.

[27] K. Sohn, H. Lee, and X. Yan, “Learning structured output representation using deep conditional generative models,” in Advances in Neural Information Processing Systems 28, Montreal, QC, Canada, Dec. 7–12, 2015, pp. 3483–3491. [Online]. Available: https://proceedings.neurips.cc/paper/2015/file/8d55a249e6baa5c06772297520da2051-Paper.pdf

[28] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models,” arXiv:2307.09288, 2023, doi: 10.48550/arXiv.2307.09288.

[29] P. K. A. Vasu et al., “FastVLM: Efficient vision encoding for vision language models,” in Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR), Nashville, TN, USA, Jun. 11–15, 2025, pp. 19769–19780. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2025/html/Vasu_FastVLM_Efficient_Vision_Encoding_for_Vision_Language_Models_CVPR_2025_paper.html

[30] Z. Xie, Z. Han, X. Wei, S. Sun, Y. Li, and J. Zhao, “S^2-VLA: State-space guided vision-language-action models for long-horizon manipulation,” arXiv:2606.27872, Jun. 2026, accepted to IJCAI-ECAI 2026, Bremen, Germany, Aug. 15–21, 2026. [Online]. Available: https://arxiv.org/abs/2606.27872

[31] S. Yang, B. Zhang, J. Zhang, Z. Tao, J. Guo, D. Ran, X. Bian, and Q. Li, “FabriVLA: A lightweight vision-language-action model with conformal action chunk uncertainty,” arXiv:2607.08575v3, Aug. 2026, doi: 10.48550/arXiv.2607.08575.

[32] Y. Zhang et al., “V-JEPA policy: Building effective world-action models on predictive visual latents,” arXiv:2609.37250, 2026, doi: 10.48550/arXiv.2609.37250.

[33] H. Zhao et al., “RynnVLA-Latent: Scaling robotic manipulation foundation models with 3D-grounded latent actions,” arXiv:2610.34356, Oct. 2026, doi: 10.48550/arXiv.2610.34356.

[34] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” in Proc. Robot.: Sci. Syst. XIX, Daegu, Republic of Korea, Jul. 10–14, 2023, doi: 10.15607/RSS.2023.XIX.016.

[35] S. Zheng, Z. Fang, Y. Du, Y. Liu, and L. Du, “SPAR: Structural pruning and action recovery for vision-language-action models,” arXiv:2610.26487, Oct. 2026, doi: 10.48550/arXiv.2610.26487.

[36] B. Zitkovich et al., “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in Proc. 7th Conf. Robot Learn. (CoRL), Atlanta, GA, USA, Nov. 6–9, 2023, Proc. Mach. Learn. Res., vol. 229, pp. 2165–2183. [Online]. Available: https://proceedings.mlr.press/v229/zitkovich23a.html

[Provide the final references following the template as part II in the Report #8]

[Formatting: Ensure that all references are listed in a uniform and correct format, following the guidelines of the citation system you have chosen (e.g., APA, MLA, Chicago, etc.). This helps ensure accuracy and ease of source verification.]

[Order: Typically, reference items are arranged in alphabetical order for easy lookup. Make sure that all sources are listed comprehensively and without omission.]

[Completeness: For each reference, provide enough information so that readers can find and review the source material. This includes the author's name, year of publication, title of the work, publisher name or journal name, and other relevant information such as page numbers or URL.]

[Accuracy: Carefully check to ensure that all information about the references is correct and up-to-date.]

[Relevance: Only include sources that were actually cited or used in your project. This helps maintain the integrity and accuracy of the project.]

[Note: The "References" section is an indispensable part of any academic project. It not only proves that you have conducted careful and grounded research but also allows future generations of students to check and expand their projects from the sources you have cited.]

IX. Appendices

[Provide the final appendices following the template as part II in the Report #9]

[If there is any supplementary information that does not fit into the main sections of the document but is still important and useful for a full understanding of the project, you can include it in the appendices. This might include raw data, source code, full versions of surveys, more detailed diagrams, or other supporting documents for your main content.]

[Note: After including all these sections, ensure that all parts of the document are logically and coherently organized, and that you have thoroughly checked for language, spelling, and grammar errors.]